



Project Progress Report

WIGMS (WEDDING INVITATION & GIFT MANAGEMENT SYSTEM)

Course	C Programming Academic Year 2025-2026
Institution	PKFIE Pkfokam Institute of Excellence
Lecturer	Mbietieu Amos Mb.
Report Type	Module Implementation Report
Files Covered	types.h ui.h ui.c
Language	C (Modular Architecture)
Date	13 March 2026
GitHub Link	https://github.com/franckkaze4/WIGMS.git

Group Members	Participation
Pettang Feumba Roddy Alann	20%
Tiofack Kaze Franck	20%
Mep Meku Audrey	20%
Tamo Taboum Israel	20%
Djatche Raziel Steevi	20%

LECTURER: MR. MBIETEU AMOS MB.

DATE: FRIDAY, 3TH APRIL 2026

TABLE OF CONTENTS	2
REPORT 1 GitHub Setup, types.h, Modules 1 & 2	3
1. Introduction	3
2. What is GitHub?	3
2.1 Prerequisites.....	3
2.2 Creating the GitHub Account.....	3
2.3 Configuring Git on Windows 11	4
2.4 Creating the Repository on GitHub	4
2.5 Cloning the Repository	4
2.6 Setting Up the Project Folder Structure.....	4
2.7 First Commit and Upload	5
2.8 Adding Teammates as Collaborators	5
2.9 Creating Branches for Each Module	5
2.10 Daily Workflow	5
2.11 Uploading Weekly Reports	6
2.12 Merging a Module into Main	6
2.13 Tracking Individual Contributions	6
3. Shared Type Definitions include/types.h	7
3.1 Purpose and Role in the Project.....	7
3.2 Include Guards	7
3.3 The Side Enumeration	7
3.4 The Person, Category and Gift Structs	8
3.5 Complete Source Code	8
4. Module 1 Person Management.....	8
4.1 Why Module 1 Was Prioritised	8
4.2 The Person Struct Design	9
4.3 Functions: create_person(), display_person(), update_person()	9
4.4 Complete Source Code person.h and person.c	9
5. Module 2 Category Management (Linked List).....	10
5.1 Objectives	10
5.2 Data Structures Used	10
5.3 File Structure	11
5.4 Functions Implemented	11
5.5 Complete Header File (category.h)	14
5.6 How to Use in Dev-C++	14
5.7 Dependencies.....	14
5.8 GitHub Contribution	15
6. Integration with Other Modules	15
7. Testing & Validation	15
8. Challenges Encountered & Solutions Applied	16
9. Conclusion Report 1	16
REPORT 2 Modules 3–5, GTK UI & New Modules 6–8	17
1. Introduction	17
2. Module 3 Priority Management (DPR Merge Sort).....	17
2.1 Objectives	17
2.2 What Is DPR?	17
2.3 Why Merge Sort for a Linked List?	18
2.4 File Structure	18

2.5 Key Functions: get_middle(), merge(), merge_sort_internal()	18
2.6 Step-by-Step Implementation Procedure	19
2.7 GitHub Contribution	19
3. Module 4 Gift Management	20
3.1 Objectives	20
3.2 Data Structure Fixed-Size Array	20
3.3 File Persistence Binary I/O (fwrite/fread)	20
3.4 Array Delete Shift Pattern	21
3.5 Step-by-Step Implementation Procedure	21
3.6 Dependencies	21
4. Module 5 Graphical User Interface (GTK+ 3.0)	22
4.1 Objectives and MVC Role	22
4.2 What Is GTK?	22
4.3 The Widget Tree Architecture	22
4.4 Signal-Callback Event Model	23
4.5 Key GTK Widgets Used (15 widgets)	23
4.6 CSS Theming Navy and Orange Palette	24
4.7 AppState Shared Application Data	24
4.8 Step-by-Step Implementation Procedure	24
5. Brainstorming Three Additional Modules	25
6. Module 6 Parking Management	25
6.1 Concept and Zone Design	25
6.2 Key Data Structures	26
6.3 New C Concept: time.h Timestamps	26
6.4 Key Functions	26
6.5 Step-by-Step Implementation Procedure	27
7. Module 7 Meal & Table Seating Management	27
7.1 Concept	27
7.2 New C Concept: Nested Struct Arrays	28
7.3 Enums Used	28
7.4 MealSummary Returning a Struct by Value	28
7.5 Step-by-Step Implementation Procedure	29
8. Module 8 Event Timeline & Schedule Management	29
8.1 Concept	29
8.2 Doubly Linked List vs Singly Linked List	30
8.3 Key Data Structures	30
8.4 Sorted Insertion Four Cases	30
8.5 File Persistence The EventSnapshot Pattern	31
8.6 Live Countdown with difftime()	31
8.7 Step-by-Step Implementation Procedure	31
9. Integration of All Eight Modules	32
9.1 Updated Project Structure	32
9.2 Updated AppState	33
9.3 Updated Makefile	33
9.4 Module Dependency Map	33
10. Challenges Encountered & Solutions Applied	34
11. Testing & Validation	34
12. GitHub Contributions	35
13. Conclusion Report 2	35

REPORT 3 The Makefile: Building and Running WIGMS.....	36
1. Introduction	36
2. What Is a Makefile?.....	36
3. Prerequisites	36
4. Complete WIGMS Makefile.....	37
5. Makefile Line-by-Line Explanation.....	37
5.1 CC Compiler Variable.....	37
5.2 CFLAGS Compiler Flags	38
5.3 LIBS Linker Libraries	38
5.4 SRC Source Files List	38
5.5 Targets: all, clean, run, install-deps.....	39
5.6 Pattern Rule %.o: %.c	39
6. Step-by-Step: How to Compile and Run.....	39
7. Common Errors and Fixes	41
8. GitHub Contribution.....	41
9. Conclusion	41

Introduction

This report documents the complete process of creating and using a GitHub repository for the WIGMS project. GitHub is a web-based platform that allows developers to store, manage, and collaborate on code. For this academic project, GitHub serves as the central location where all team members share their code, track progress, and demonstrate individual contributions to the lecturer.

This is the second weekly progress report for the WIGMS (Wedding Invitation and Gift Management System) project. Report 1 documented the GitHub repository setup, the shared types.h library, Module 1 (Person Management), and Module 2 (Category Management). This report continues from where Report 1 ended and documents the implementation of Module 3 (Priority Management DPR Merge Sort), Module 4 (Gift Management with file persistence), and Module 5 (Graphical User Interface using the GTK+ 3.0 library). It also presents the brainstorming and initial implementation of three additional modules proposed to extend the system: Module 6 (Parking Management), Module 7 (Meal and Table Seating), and Module 8 (Event Timeline and Schedule).

System Description

The system designed is a Wedding Management System. It allows a Wedding Planner to manage different aspects such as persons, categories, priorities, gifts, and navigation menus.

Actor Description

Wedding Planner: The main user who interacts with the system to perform all operations.

Use Cases Description

1. Manage Persons: Add, update, and view persons involved in the wedding.
2. Manage Categories: Organize persons into categories.
3. Manage Priorities: Set priorities for categories or tasks.
4. Manage Gifts: Record and manage gifts.
5. Navigate Menus: Access different parts of the system.

Relationships

Association: Connects the Wedding Planner to all use cases.

Include Relationship:

- Manage Categories includes Manage Persons.
- Manage Priorities includes Manage Categories.

These relationships indicate dependency between functionalities.

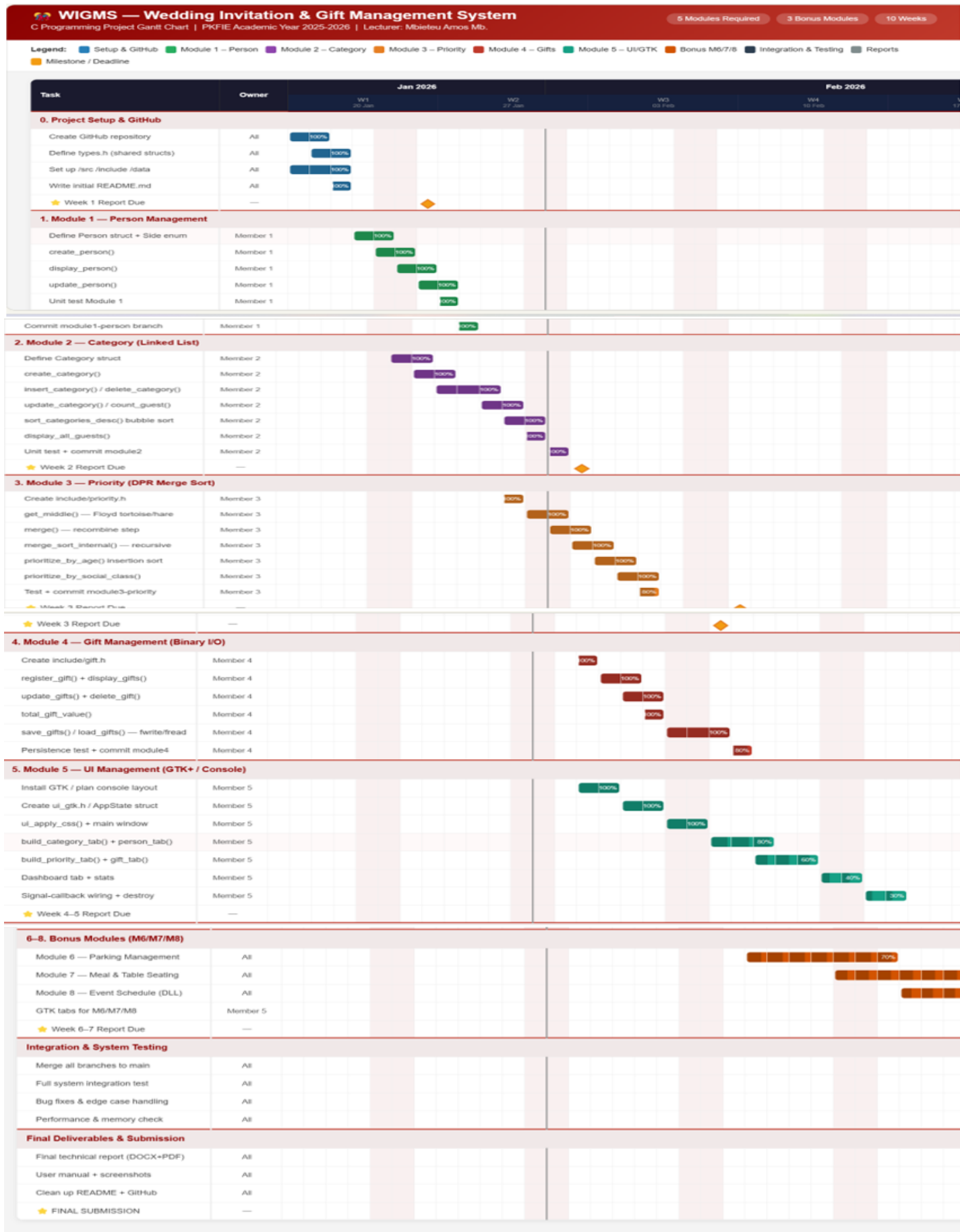
Procedure

1. Open StarUML.
2. Create a new Use Case Diagram.
3. Add an Actor named "Wedding Planner".
4. Add all use cases.
5. Connect the actor using associations.
6. Add «include» relationships.
7. Add system boundary.
8. Format diagram in black and white.

Result

A complete Use Case Diagram was successfully created representing the Wedding System. The diagram clearly shows all interactions and dependencies between components.

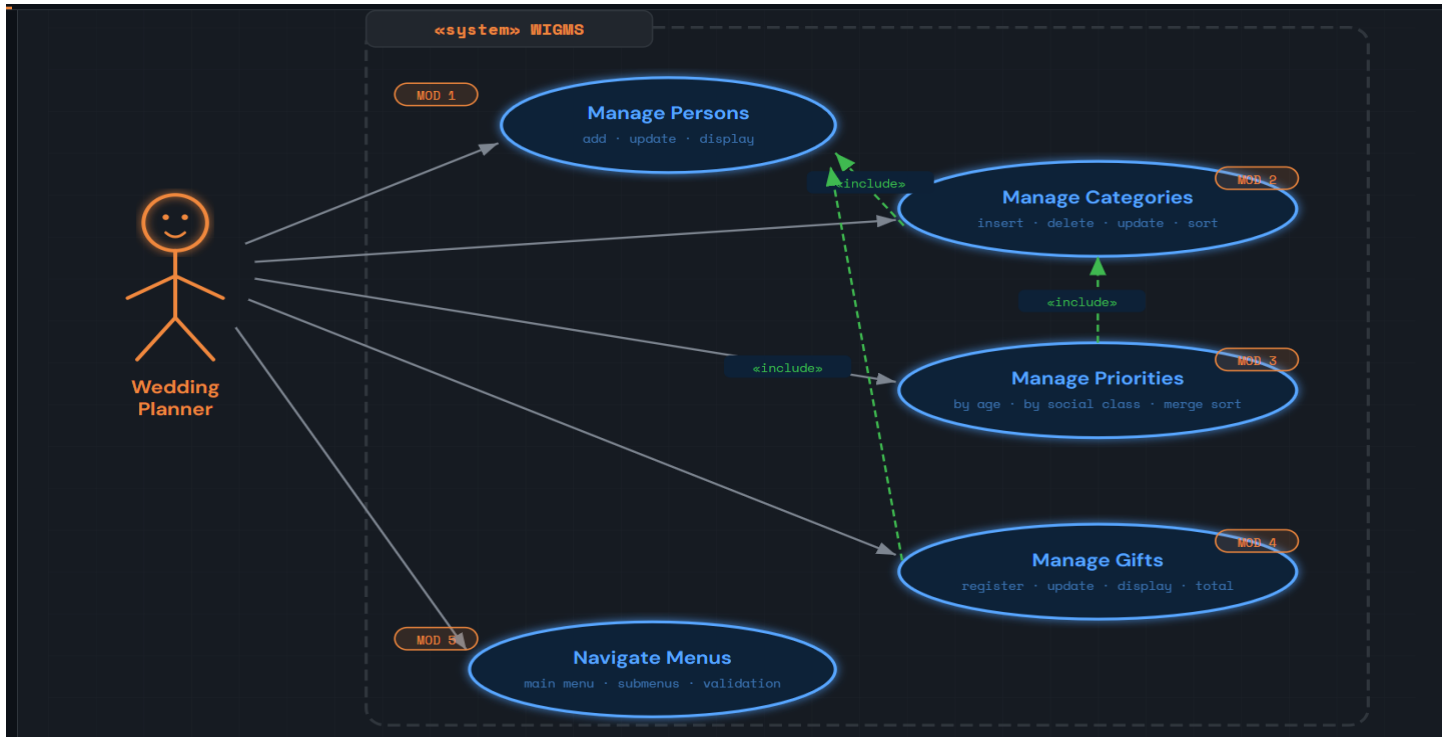
GANTT CHART



2. Class Diagram

2.1 Objective

The Class Diagram answers the question: How is the data structured? It models all the C structs defined in the project as UML classes, showing their attributes (fields), operations (functions) and the relationships between them. This diagram is the foundation of the object model and directly corresponds to the C code structures.



2.2 Classes Identified

Three main classes were identified from the project requirements, each corresponding to a C struct:

2.2.1 Person Class (Module 1)

The Person class models a wedding guest. It is defined by the following C struct in the source code:

Attribute	Type	Visibility	Description
id	int	Private (-)	Unique identifier for each person
name	char[100]	Private (-)	Full name of the guest
age	int	Private (-)	Age of the guest
social_class	char[50]	Private (-)	Social category of the guest
side	Side	Private (-)	Wedding side: LE (Groom) or LA (Bride)

Operations defined for the Person class:

- `create_person()` Creates and initialises a new Person instance from user input.
- `display_person(p)` Displays all attributes of a given Person on screen.
- `update_person(*p)` Updates the fields of an existing Person using a pointer.

2.2.2 The Side Enumeration

The Side type is modelled as a UML enumeration class. It represents the wedding side to which each guest belongs and can only hold one of two values:

- LE The Groom's side (Le marie in French)
- LA The Bride's side (La mariee in French)

In the class diagram, Person has a dependency relationship (<<use>>) pointing to Side, meaning the side attribute in Person uses Side as its type. This matches the C enum definition exactly.

2.2.3 Category Class (Module 2)

The Category class implements a singly linked list of guest categories. Each category holds up to 4 Person objects and a pointer to the next category node, forming the linked list structure.

Attribute	Type	Visibility	Description
id	int	Private (-)	Unique identifier for the category
code	char[50]	Private (-)	Category code or name
guests	Person[4]	Private (-)	Array of up to 4 Person objects
next	*Category	Private (-)	Pointer to the next node in the linked list

Operations defined for the Category class:

- create_category() Allocates and initialises a new Category node.
- insert_category(**head) Inserts a new category into the linked list.
- delete_category(**head, id) Removes a category by its identifier.
- update_category(*head, id) Updates an existing category's fields.
- count_guest(*head) Counts the total number of guests across all categories.
- sort_categories_desc(**head) Sorts categories in descending order.
- display_all_guests(*head) Displays all guests in all categories.

2.2.4 Gift Class (Module 4)

The Gift class models a wedding gift given by a guest. Each gift is linked to a specific guest through the guest_id attribute.

Attribute	Type	Visibility	Description
gift_id	int	Private (-)	Unique identifier for the gift
name	char[100]	Private (-)	Name or description of the gift
value	float	Private (-)	Monetary value of the gift
guest_id	int	Private (-)	ID of the guest who gave the gift

Operations defined for the Gift class:

- register_gift() Registers a new gift entry linked to a guest.
- display_gifts() Displays all recorded gifts.
- update_gifts() Modifies an existing gift record.
- total_gift_value() Calculates and returns the total value of all gifts.
- save_gifts() Saves all gift records to a file for persistence.
- load_gifts() Loads previously saved gift records from file.

2.3 Relationships Between Classes

Three types of relationships were identified and drawn in the class diagram:

Relationship	From	To	Type	Multiplicity	Meaning
Composition	Category	Person	Filled diamond	1 to 4	A Category contains exactly 4 Person objects

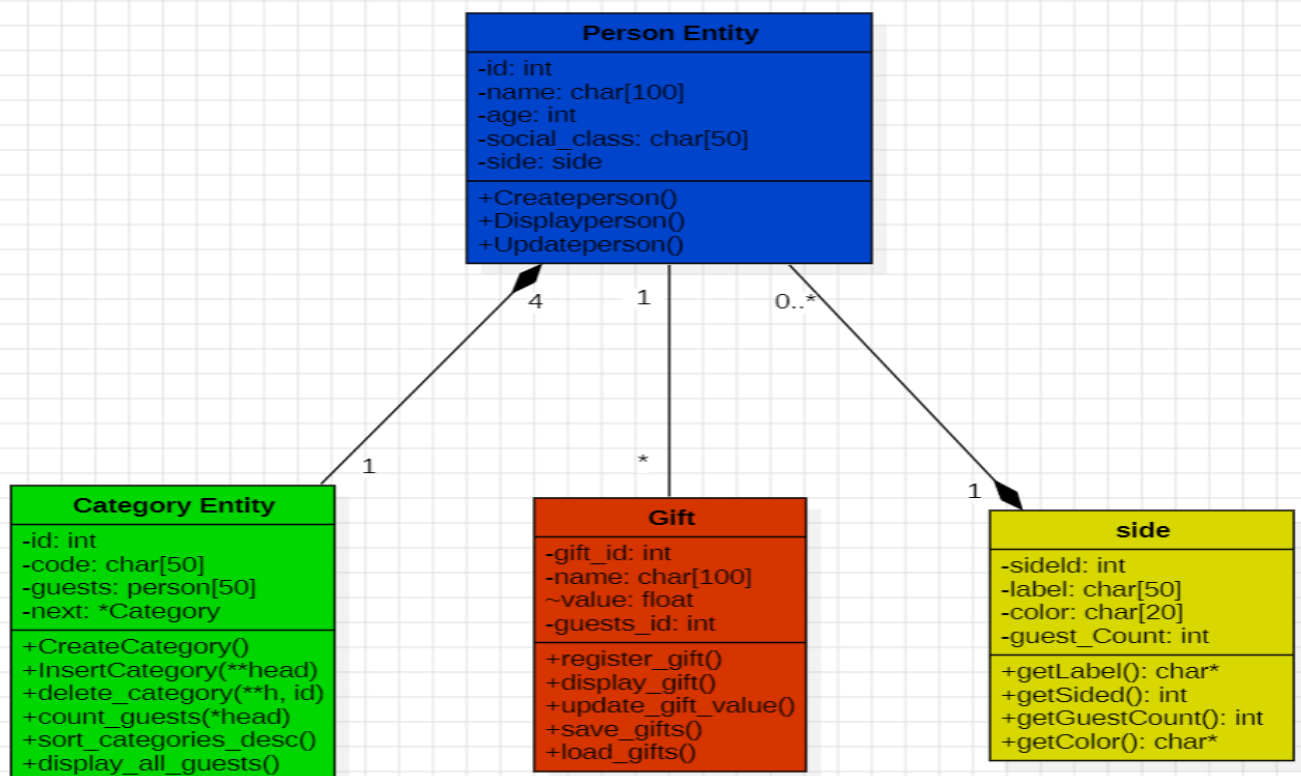
Association	Gift	Person	Plain arrow	* to 1	Many Gifts are linked to one Person (guest)
Dependency	Person	Side	Dashed arrow <<use>>	N/A	Person uses Side as the type of its side attribute
Self-link	Category	Category	Dashed arrow (*next)	0..*	Category points to next node (linked list)

The composition between Category and Person is the most important relationship: it means that a Category owns its Persons if the Category is deleted, its guest entries are also removed. This is consistent with the C implementation where guests[] is a fixed array inside the struct.

2.4 How the Class Diagram Was Drawn in StarUML

The following steps were followed to produce the class diagram using StarUML:

1. Created a new project and added a Class Diagram canvas via Model > Add Diagram > Class Diagram.
2. Added three class boxes from the Toolbox: Person, Category, and Gift.
3. Added an Enumeration box for Side with values LE and LA.
4. Filled in attributes for each class using right-click > Add > Attribute, specifying name and type in the Properties panel.
5. Added operations using right-click > Add > Operation for each function.
6. Set visibility to Private (-) for all attributes and Public (+) for all operations.
7. Drew a Composition arrow from Category to Person using the Toolbox.
8. Drew a Dependency (dashed) arrow from Person to Side and labelled it <<use>>.
9. Drew an Association arrow from Gift to Person and set multiplicity to * and 1.
10. Added a self-association on Category to represent the *next linked list pointer.



3. Sequence Diagram

2.1 Objective

The Sequence Diagram answers the question: In what order do the modules call each other? It shows the flow of messages between the user and the system modules for one specific scenario, tracing each function call step by step from top to bottom. Time flows downward the first message at the top happens first.

2.2 Scenario Chosen: Register a Gift

The scenario chosen to illustrate the Sequence Diagram is: Register a Gift for a Guest. This scenario was selected because it involves the most modules communicating together the UI, the Gift module, the Person module and the file system making it the richest and most representative interaction in the system.

2.3 Participants (Lifelines)

Five participants were placed across the top of the diagram, each with a vertical dashed lifeline running downward:

Participant	Represents	Module	Role
User	The wedding planner at the keyboard	External actor	Initiates all actions
ui.c	The user interface module	Module 5	Receives input, calls menus
gift.c	The gift management module	Module 4	Processes gift registration
person.c	The person management module	Module 1	Validates and returns guest data
gifts.dat	The file used to save gift records	File system	Persists data between sessions

2.4 Messages Exchanged

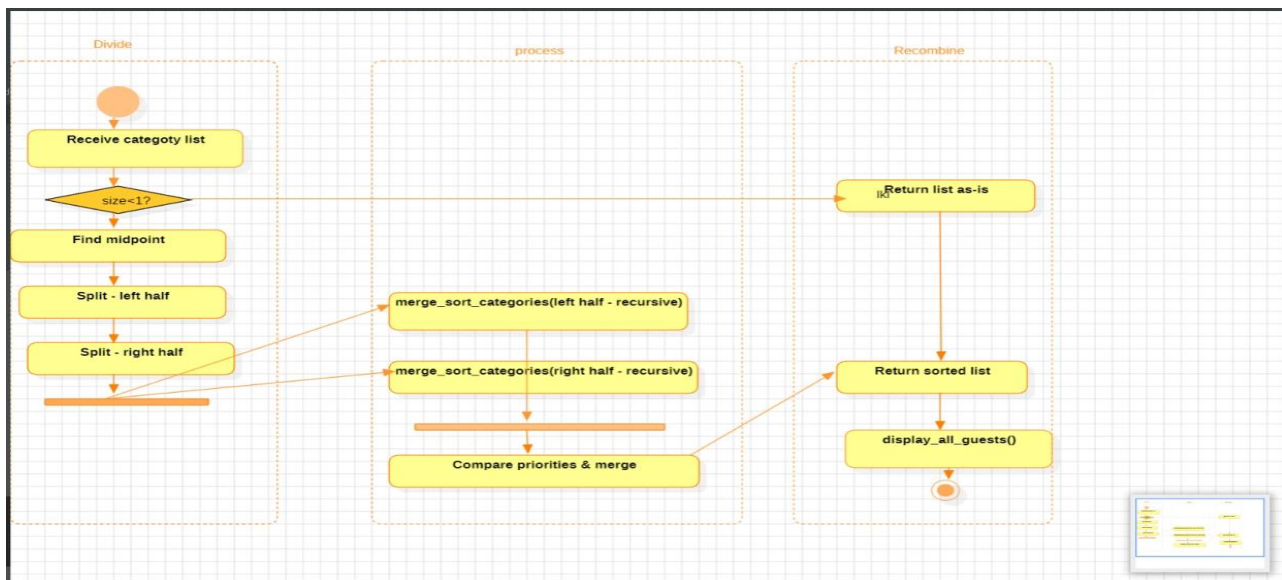
Ten messages were identified and drawn as horizontal arrows between the lifelines. Solid arrows represent function calls (going left to right). Dashed arrows represent return values (going right to left).

2.5 Key UML Notations Used

- Lifelines vertical dashed lines below each participant box, representing the participant existing over time.
- Activation bars thin rectangles on a lifeline showing when a module is actively executing a function.
- Synchronous messages solid filled arrowhead, meaning the sender waits for a response before continuing.
- Return messages dashed arrow with open arrowhead, carrying the result back to the caller.
- The scenario is framed inside a sd (sequence diagram) box labelled: sd: Register a Gift for a Guest.

2.6 How It Was Drawn in StarUML

1. Added a Sequence Diagram canvas via Model > Add Diagram > Sequence Diagram.
2. Created five Lifeline elements from the Toolbox, labelled User, ui.c, gift.c, person.c and gifts.dat.
3. Drew Message arrows between lifelines by clicking and dragging from one lifeline to another.
4. Labelled each arrow with the function name by double-clicking on the arrow.
5. Added Return Message arrows (dashed) for each function response.
6. Added Activation bars by selecting each lifeline and enabling the activation region in Properties.
7. Wrapped the whole diagram in a Combined Fragment box of type sd.



4. Activity Diagram

3.1 Objective

The Activity Diagram answers the question: How does a process flow from start to finish? It models a process as a sequence of actions connected by control flows, including decision points (if/else), parallel branches (fork/join) and loops. It is similar to a flowchart but richer, because it supports concurrent actions and swimlanes.

3.2 Process Chosen: DPR Merge Sort (Module 3)

The process chosen to illustrate the Activity Diagram is the DPR Merge Sort algorithm from Module 3 (Priority Management). DPR stands for Divide, Process, Recombine a recursive algorithm that sorts the category linked list by guest priority (age or social class). This process was chosen because it is the most algorithmically complex operation in the project and has a clear multi-phase structure perfectly suited for an activity diagram.

3.3 Swimlanes Used

The diagram is divided into three vertical swimlanes, one per phase of the DPR algorithm. Each action is placed in the swimlane of the phase responsible for it:

3.4 Actions and Nodes

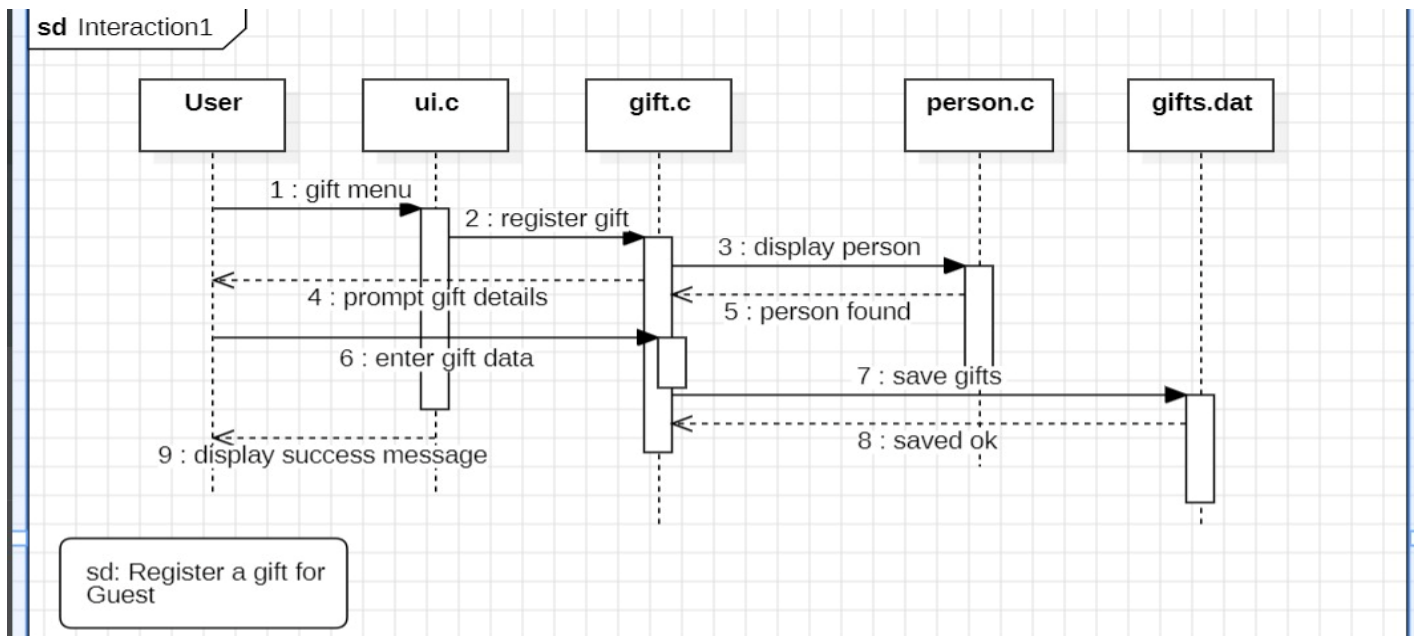
The following UML activity nodes were used in the diagram:

Node Type	Symbol	Used for	Example in diagram
Initial node	Filled black circle	Start of the process	Start of merge sort
Action	Rounded rectangle	One processing step	Find midpoint, Split left half
Decision	Diamond	A condition with two paths	Is list size <= 1?
Fork bar	Thick horizontal bar	Splits flow into parallel branches	Sort left AND right half at same time
Join bar	Thick horizontal bar	Merges parallel branches back	Wait for both halves to finish
Final node	Circle with filled dot inside	End of the process	End after display_all_guests()

3.5 Full Flow Description

The complete step-by-step flow of the Activity Diagram is as follows :

1. Start node the merge sort process begins.
2. Receive category list the function receives the linked list head pointer as input.
3. Decision diamond: Is list size ≤ 1 ? this is the base case of the recursion.
4. If YES (base case): the flow goes directly to the Recombine swimlane Return list as-is, then display_all_guests(), then End.
5. If NO (recursive case): the flow continues in the Divide swimlane.
6. Find midpoint calculate the middle of the list.
7. Split into left half separate the first half of the list.
8. Split into right half separate the second half of the list.
9. Fork bar the flow splits into two parallel branches in the Process swimlane.
10. Branch 1: merge_sort_categories(left half) recursive call on the left half.
11. Branch 2: merge_sort_categories(right half) recursive call on the right half.
12. Join bar both branches complete and the flow merges back together.
13. Compare priorities and merge combine the two sorted halves in the correct order.
14. Return sorted list pass the result to the Recombine swimlane.
15. display_all_guests() show the final sorted list.
16. Final node the process ends.



5. Component Diagram

3.1 Objective

The Component Diagram answers the question: How is the software organised into files? While the Class Diagram shows the data model, the Component Diagram shows the physical architecture of the application the five C source files, how they depend on each other, and how they connect to external systems such as the MySQL database and the gifts.dat file.

This diagram is directly linked to the MVC (Model-View-Controller) architecture recommended in the project brief: Modules 1-4 form the Model layer, Module 5 (ui.c) forms the Controller layer, and the console or GUI forms the View layer.

3.2 Components Identified

Five components were identified, one for each .c source file in the project:

Component	File	Module	Layer	Responsibility
person.c	person.c + person.h	Module 1	Model	Manages Person struct creation, display, update
category.c	category.c + category.h	Module 2	Model	Manages Category linked list insert, delete, sort
priority.c	priority.c + priority.h	Module 3	Model	DPR Merge Sort prioritise guests by age and class
gift.c	gift.c + gift.h	Module 4	Model	Manages Gift records register, save, load, total
ui.c	ui.c + ui.h	Module 5	Controller	Handles all menus, input validation and user interaction

Two external systems were also included in the diagram:

- MySQL the database management system used for persistent data storage.
- gifts.dat a binary or text file used by gift.c for saving and loading gift records.

3.3 Interfaces

In UML component diagrams, interfaces define the services that components offer or require. Two types of interface notation were used:

3.3.1 Provided Interfaces (Ball notation)

A small circle attached to the right side of a component box means the component provides a service that other components can use. The following provided interfaces were defined:

- IPersonService provided by person.c (offers create, display, update functions)
- ICategoryService provided by category.c (offers insert, delete, sort, display functions)
- IPriorityService provided by priority.c (offers merge sort and prioritisation functions)
- IGiftService provided by gift.c (offers register, save, load, total functions)

3.3.2 Required Interfaces (Socket notation)

A half-circle attached to the left side of a component means the component needs a service from another component. The following required interfaces were identified:

- category.c requires IPersonService because Category contains Person objects, it needs to call Person functions.
- priority.c requires ICategoryService because Priority sorts Category nodes, it needs to access Category functions.
- gift.c requires IPersonService because each Gift is linked to a guest_id, it needs to access Person data.

3.4 Dependencies Between Components

The following dependency relationships (<<use>> arrows dashed lines with arrowheads) were drawn in the diagram:

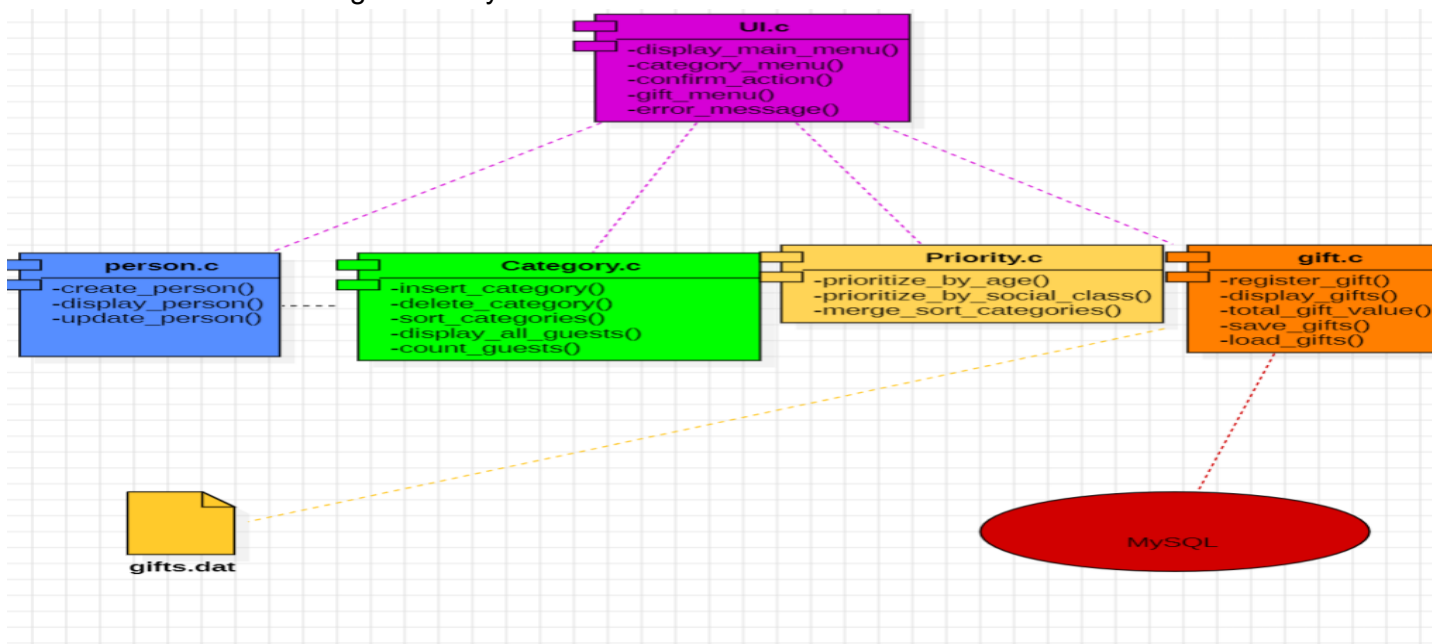
From	To	Arrow Type	Reason
ui.c	person.c	<<use>> dashed arrow	UI calls person_menu() which calls Person functions
ui.c	category.c	<<use>> dashed arrow	UI calls category_menu() which calls Category functions
ui.c	priority.c	<<use>> dashed arrow	UI calls priority_menu() which calls Priority functions
ui.c	gift.c	<<use>> dashed arrow	UI calls gift_menu() which calls Gift functions
category.c	person.c	<<use>> dashed arrow	Category uses Person as it stores Person objects inside
priority.c	category.c	<<use>> dashed arrow	Priority sorts Category nodes in the linked list
gift.c	person.c	<<use>> dashed arrow	Gift is linked to a Person via guest_id
gift.c	MySQL	<<call>> dashed arrow	Gift module queries MySQL for persistent storage
gift.c	gifts.dat	<<persist>> dashed arrow	Gift module reads and writes to gifts.dat file

The most important observation from this diagram is that ui.c is the only component that depends on ALL other modules. This confirms its role as the Controller in the MVC architecture it orchestrates all calls but contains no business logic itself.

3.5 How the Component Diagram Was Drawn in StarUML

The following steps were followed to produce the component diagram in StarUML:

11. Created a new canvas via Model > Add Diagram > Component Diagram.
12. Added five Component boxes from the Toolbox (one per .c file), labelled with the filename and stereotype <<component>>.
13. Added two external components: MySQL (labelled <<database>>) and gifts.dat (labelled <<artifact>>).
14. For each component, right-clicked and used Add > Port to add interface attachment points.
15. Added provided interface balls on the right side of person.c, category.c, priority.c and gift.c.
16. Added required interface sockets on the left side of category.c, priority.c and gift.c.
17. Drew Dependency arrows (dashed lines with <<use>> labels) from ui.c to all four model components.
18. Drew additional dependency arrows between modules that call each other (category to person, priority to category, gift to person).
19. Drew dependency arrows from gift.c to MySQL and to gifts.dat.
20. Added module badges and layer annotations to make the MVC structure visible.



2. What is GitHub?

GitHub is an online platform built on top of Git, a version control system. It allows multiple developers to work on the same project at the same time without overwriting each other's work. Every change made to the code is recorded with a timestamp and the name of the person who made it.

The key benefits of using GitHub for the WIGMS project are:

- Each team member can work on their own module independently
- The lecturer can see who contributed what and when
- All weekly reports and code files are stored safely online
- Previous versions of code can be recovered at any time
- The repository serves as a professional portfolio item

3. Prerequisites

Before creating the repository, the following tools were installed and configured:

Tool	Purpose
GitHub Account	Required to create and manage the online repository
Git for Windows	Allows running Git commands in the Windows 11 terminal
Windows 11 Terminal	Used to type all Git commands to manage the project
Dev-C++	Used to write and test the C code for each module

4. Creating the GitHub Account

The first step was to create a personal GitHub account by visiting github.com and signing up. The account was created with the username `franckkaze4`. This username appears on all commits and identifies the contributor throughout the project.

5. Configuring Git on Windows 11

After installing Git for Windows, the terminal was opened and Git was configured with the team member's identity using the following commands:

```
git config --global user.name "franckkaze4"
git config --global user.email "franckkaze4@gmail.com"
```

This step is important because every commit made will carry this name and email, allowing the lecturer to identify who made each contribution.

6. Creating the Repository on GitHub

The repository was created directly on the GitHub website by following these steps:

1. Logged into github.com with username `franckkaze4`
2. Clicked the + button in the top right corner
3. Selected New Repository
4. Entered the repository name: `WIGMS`
5. Added the description: `Wedding Invitation and Gift Management System - C Programming Project`
6. Set visibility to Public so the lecturer can view it
7. Checked the Add a README file option
8. Clicked Create Repository

7. Cloning the Repository to the Computer

After creating the repository online, it was downloaded to the local computer using the clone command in the Windows 11 terminal:

```
git clone https://github.com/franckkaze4/WIGMS.git
cd WIGMS
```

This created a copy of the repository on the computer inside a folder named WIGMS, allowing work to be done locally before uploading to GitHub.

8. Setting Up the Project Folder Structure

The required folder structure specified in the project PDF was created using the following commands in PowerShell:

```
mkdir src
mkdir include
mkdir data
mkdir reports
New-Item -Path src\.gitkeep -ItemType File
New-Item -Path include\.gitkeep -ItemType File
New-Item -Path data\.gitkeep -ItemType File
New-Item -Path reports\.gitkeep -ItemType File
```

The .gitkeep files are empty placeholder files that allow Git to track empty folders, since Git does not track folders by default.

The resulting folder structure is:

```
/WIGMS
  /src      <- all .c source files
  /include  <- all .h header files
  /data     <- saved data files
  /reports  <- weekly reports (DOCX + PDF)
  README.md <- project description
```

9. First Commit and Upload

Once the folder structure was created, the changes were saved and uploaded to GitHub using the following three commands:

```
git add .
git commit -m "Initial project structure"
git push origin main
```


These three commands are the core workflow of GitHub and are used every time new work needs to be saved online. Their meaning is:

Command	What It Does
<code>git add .</code>	Selects all modified files and prepares them to be saved
<code>git commit -m "..."</code>	Saves the changes locally with a descriptive message
<code>git push origin main</code>	Uploads the saved changes to the GitHub website

10. Adding Teammates as Collaborators

Since the project requires up to 5 students per group, teammates were added as collaborators by following these steps on the GitHub website:

9. Went to the WIGMS repository page on github.com
10. Clicked Settings in the top menu
11. Clicked Collaborators in the left sidebar
12. Clicked Add people
13. Entered each teammate's GitHub username
14. Each teammate received an email invitation and accepted it

Once added, each teammate cloned the repository to their own computer using:

```
git clone https://github.com/franckkaze4/WIGMS.git
```

11. Creating Branches for Each Module

To avoid conflicts between team members working on different modules, each student created their own branch. A branch is an independent copy of the project where a student can work without affecting the main code.

```
git checkout -b module1-person
git checkout -b module2-category
git checkout -b module3-priority
git checkout -b module4-gifts
git checkout -b module5-ui
```

Branch Name	Student	Module
-------------	---------	--------

Module1-person	MEPMEKU LARISSA AUDREY	Person Management
Module2-category	TIOFACK KAZE FRANCK (GROUP LEADER)	Category Management
Module3-priority	TAMO TABOUM ISRAEL	Priority Management
Module4-gifts	DJATCHE DOMFANG RAZIEL STEEVI	Gift Management
Module5-ui	PETTANG FEUMBA RODDY ALANN	User Interface

12. Daily Workflow

Every day before starting work, the latest version of the project is pulled from GitHub to ensure the local copy is up to date:

```
git pull origin main
```

After writing or editing code, the work is saved and uploaded using:

```
git add .  
git commit -m "Module 2: describe what was done"  
git push origin module2-category
```

13. Uploading Weekly Reports

Weekly reports in DOCX and PDF format are placed in the reports folder and uploaded to GitHub using the same workflow:

```
git add reports\week1_report.docx  
git add reports\week1_report.pdf  
git commit -m "Added Week 1 progress report"  
git push origin module2-category
```

14. Merging a Module into Main

When a module is fully completed and tested, it is merged into the main branch through a Pull Request on the GitHub website:

15. Go to the WIGMS repository on github.com

-
16. Click Pull Requests in the top menu
 17. Click New Pull Request
 18. Select the module branch (e.g. module2-category)
 19. Write a description of what was completed
 20. Click Create Pull Request
 21. A teammate reviews the code
 22. Click Merge Pull Request to add it to main

15. Tracking Individual Contributions

Since the project requires each student to show their participation percentage, GitHub automatically tracks this. The lecturer can visit the repository and click Insights then Contributors to see exactly how many commits each person made, what files they changed, and when they worked. This makes individual contributions fully transparent.

16. Conclusion

The GitHub repository for the WIGMS project has been successfully created and configured. The repository at github.com/franckkaze4/WIGMS contains the full modular project structure with separate branches for each module. The team is now able to work collaboratively, upload weekly reports, track individual contributions, and maintain a professional portfolio-ready repository as required by the project specifications.

2. Shared Type Definitions `include/types.h`

2.1 Purpose and Role in the Project

`types.h` is the single source of truth for all data structures used in WIGMS. It is included by every other header file in the project. By centralising all struct and enum definitions in one place, we guarantee type consistency across all modules: when Module 2 creates a Category node containing Person guests, and Module 1 operates on those same Person records, both modules are working with byte-for-byte identical type definitions.

Without `types.h`, each module would independently define its own version of Person or Gift. Even with identical field names and types, the C compiler would treat them as distinct incompatible types, making cross-module function calls impossible.

2.2 Include Guards

The file opens and closes with preprocessor include guards:

```
#ifndef TYPES_H
#define TYPES_H
/* ... content ... */
#endif /* TYPES_H */
```

In a multi-file project, the compiler processes each translation unit (.c file) independently. If two .c files both include types.h, and types.h has no guard, the compiler sees each struct definition twice in the same translation unit and raises a 'redefinition' error. The guard works as follows: the first time the compiler encounters `#ifndef TYPES_H`, the macro is undefined so the guard passes and the content is compiled. `TYPES_H` is then immediately defined. Every subsequent inclusion finds `TYPES_H` already defined, the `#ifndef` fails, and the entire file content is skipped.

2.3 The Side Enumeration

```
typedef enum {
    LE,      /* Groom's side  Le marié */
    LA       /* Bride's side  La mariée */
} Side;
```

An enumeration assigns symbolic names to integer constants. `LE = 0` and `LA = 1` at the machine level. Using an enum instead of plain integers makes the code self-documenting: reading `'p.side == LE'` is immediately meaningful, whereas `'p.side == 0'` requires the reader to remember what 0 represents. The typedef keyword removes the need to write `'enum Side'` every time we simply write `'Side'`.

2.4 The Person, Category, Gift Struct

Person groups five related fields into a single named type. The char arrays `name[100]` and `social_class[50]` are fixed-length string buffers this means each Person struct always occupies exactly the same number of bytes in memory (important for binary file I/O in Module 4). The side field uses the Side enum defined above, demonstrating that one typedef can build on another.

Category is the linked list node. The field `'struct Category *next'` is a self-referential pointer it points to another node of the same type, forming the chain. We must write `'struct Category *next'` and not simply `'Category *next'` because the typedef alias `'Category'` is not yet complete at the point where next is declared inside the struct body. The compiler needs the full struct tag to understand what the pointer points to.

The `guests[4]` field embeds an array of up to four Person structs directly inside the node no additional allocation is required for the guests themselves. `guest_count` tracks how many of the four slots are actually populated.

Gift uses float for monetary value to support decimal amounts (e.g., 15000.50 FCFA). The `guest_id` field creates a logical foreign-key relationship between a gift and the guest who offered it, mirroring relational database design principles. The `MAX_GIFTS` macro (200) defines the maximum size of the gifts array allocated in main.c.

2.7 Complete Source Code

```

[*] types.h
2  #define TYPES_H
3  /* =====
4  WIGMS - Shared Data Types
5  All structs and enums used across the entire application
6  ===== */
7
8  /* Which side of the family the guest belongs to:
9  LE = Le marié (groom's side)
10 LA = La mariée (bride's side) */
11 typedef enum {
12     LE,
13     LA
14 } Side;
15
16 /* A single wedding guest / person */
17 typedef struct {
18     int id;
19     char name[100];
20     int age;
21     char social_class[50]; /* e.g. "VIP", "Family", "Friend" */
22     Side side;
23 } Person;
24
25 /* A guest category - linked list node.
26 Each node holds up to 4 guests. */
27 typedef struct Category {
28     int id;
29     char code[50]; /* e.g. "FAM_LE", "AMI_LA" */
30     int guest_count; /* how many of the 4 slots are filled */
31     Person guests[4];
32     struct Category *next;
33 } Category;
34
35 /* A gift registered for the wedding */
36 typedef struct {
37     int gift_id;
38     char name[100];
39     float value;
40     int guest_id; /* which guest gave this gift */
41 } Gift;
42
43 #define MAX_GIFTS 200
44 #endif /* TYPES_H */

```

1. MODULE 1

WHY MODULE 1 WAS PRIORITISED

Within the five-module architecture of this project, Module 1-person management -is the foundational data layer of the entire application. It was the first module to be designed and the last to be modified, since every other module depends on the person struct it defines.

The person struct was designed to capture all attributes relevant to a wedding guest in the Cameroon context:

- Id
- Name [100]
- Age
- Social class [50]
- Side

main.c person.h.gch person.h [*] person.c person.o Project-Restaurant_Software_python.pdf Untitled1.cpp

```

1  #include <stdio.h>
2  #include "person.h"
3
4  Person create_person() {
5      Person p;
6
7      printf("Enter ID: ");
8      scanf("%d", &p.id);
9
10     printf("Enter name: ");
11     scanf("%s", p.name);
12
13     printf("Enter age: ");
14     scanf("%d", &p.age);
15
16     printf("Enter social class: ");
17     scanf("%s", p.social_class);
18
19     int s;
20     printf("Side (0 = LE, 1 = LA): ");
21     scanf("%d", &s);
22
23     p.side = s;
24
25     return p;
26 }
27
28 void display_person(Person p) {
29     printf("\nID: %d\n", p.id);
30     printf("Name: %s\n", p.name);
31     printf("Age: %d\n", p.age);
32     printf("Class: %s\n", p.social_class);

```

```

32     printf("Class: %s\n", p.social_class);
33 }
34
35 void update_person(Person *p) {
36     printf("New name: ");
37     scanf("%s", p->name);
38
39     printf("New age: ");
40     scanf("%d", &p->age);
41 }

```

main.c person.h.gch person.h [*] person.c person.o Project-Restaurant_Software_python.pdf Untitled1.cpp

```

1  #ifndef PERSON_H
2  #define PERSON_H
3
4  typedef enum { LE, LA } Side;
5
6  typedef struct {
7      int id;
8      char name[100];
9      int age;
10     char social_class[50];
11     Side side;
12 } person;
13
14 person create_person();
15 void display_person(person p);
16 void update_person(person *p);
17
18 #endif

```

2. Module 2

The main objectives of Module 2 are:

- Implement a dynamic linked list to manage guest categories
- Provide full CRUD operations: Create, Read, Update and Delete
- Count the total number of guests across all categories
- Sort categories in descending order by ID
- Display all guests across every category in the list

3. Data Structures Used

3.1 The Person Struct (from Module 1)

Since each category contains guests, Module 2 depends on the Person struct defined in Module 1:

```
typedef enum { LE, LA } Side;

typedef struct {
    int id;
    char name[100];
    int age;
    char social_class[50];
    Side side;
} Person;
```

3.2 The Category Struct

The Category struct forms the main node of the linked list. Each node holds up to 4 guests and a pointer to the next category:

```
typedef struct Category {
    int id;
    char code[50];
    Person guests[4];
    struct Category *next;
} Category;
```

Field descriptions:

- `id` Unique identifier for the category
- `code[50]` Short label or code name for the category (e.g. VIP, FAMILY)
- `guests[4]` Array of up to 4 Person structs belonging to this category
- `*next` Pointer to the next Category node in the linked list

4. File Structure

Module 2 is composed of two files following the modular C architecture required by the project:

File	Role
include/category.h	Header file contains struct definition and function declarations
src/category.c	Source file contains the actual implementation of all functions
include/person.h	Dependency required because Category contains Person guests[]

5. Functions Implemented

5.1 create_category()

Creates a new Category node dynamically using malloc(), prompts the user for an ID and code, sets the next pointer to NULL, and returns the new node.

```
Category* create_category() {
    Category *c = (Category*)malloc(sizeof(Category));
    printf("Enter Category ID: ");
    scanf("%d", &c->id);
    printf("Enter Category Code: ");
    scanf("%s", c->code);
    c->next = NULL;
    return c;
}
```

5.2 insert_category()

Inserts a new category at the head of the linked list. This is the most efficient insertion method as it runs in $O(1)$ time.

```
void insert_category(Category **tete) {
    Category *new = create_category();
    new->next = *tete;
    *tete = new;
    printf("Category inserted successfully.\n");
}
```

5.3 delete_category()

Traverses the linked list to find a category by ID, then removes it by relinking the previous node to the next node, and frees the deleted node from memory.

```
void delete_category(Category **tete, int id) {
    Category *current = *tete, *previous = NULL;
    while (current != NULL) {
        if (current->id == id) {
            if (previous == NULL) *tete = current->next;

```

```

        else previous->next = current->next;
        free(current);
        printf("Category deleted successfully.\n");
        return;
    }
    previous = current;
    current = current->next;
}
printf("Category not found.\n");
}

```

5.4 update_category()

Searches for a category by ID and updates its code field with a new value provided by the user.

```

void update_category(Category *tete, int id) {
    while (tete != NULL) {
        if (tete->id == id) {
            printf("Enter new code: ");
            scanf("%s", tete->code);
            printf("Category updated successfully.\n");
            return;
        }
        tete = tete->next;
    }
    printf("Category not found.\n");
}

```

5.5 count_guest()

Traverses the entire linked list and counts the total number of guests. Since each category holds exactly 4 guests, it multiplies the number of categories by 4.

```

int count_guest(Category *tete) {
    int count = 0;
    while (tete != NULL) {
        count += 4;
        tete = tete->next;
    }
    return count;
}

```

5.6 sort_categories_desc()

Sorts all categories in descending order by their ID using a bubble sort algorithm. It repeatedly swaps adjacent nodes whose IDs are in the wrong order until no more swaps are needed.

```

void sort_categories_desc(Category **tete) {
    if (*tete == NULL) return;
    int swapped;
    Category *current;
    do {
        swapped = 0;
        current = *tete;
        while (current->next != NULL) {
            if (current->id < current->next->id) {

```

```

        int temp = current->id;
        current->id = current->next->id;
        current->next->id = temp;
        swapped = 1;
    }
    current = current->next;
}
} while (swapped);
}

```

5.7 display_all_guests()

Traverses the linked list from head to tail and prints the ID and code of every category found.

```

void display_all_guests(Category *tete) {
    while (tete != NULL) {
        printf("Category ID: %d  Code: %s\n",
            tete->id, tete->code);
        tete = tete->next;
    }
}

```

6. Complete Header File (category.h)

The header file uses include guards to prevent double inclusion and declares all functions for use by other modules:

```

#ifndef CATEGORY_H
#define CATEGORY_H

#include "person.h"

typedef struct Category {
    int id;
    char code[50];
    Person guests[4];
    struct Category *next;
} Category;

Category* create_category();
void insert_category(Category **tete);
void delete_category(Category **tete, int id);
void update_category(Category *tete, int id);
int count_guest(Category *tete);
void sort_categories_desc(Category **tete);
void display_all_guests(Category *tete);

#endif

```

7. How to Use in Dev-C++

Follow these steps to add Module 2 to a Dev-C++ project:

- Open Dev-C++ and create a new Console Application C Project named WIGMS

- Create the file include\person.h and paste the Person struct and function declarations
- Create the file include\category.h and paste the Category struct and function declarations
- Create the file src\category.c and paste all 7 function implementations
- In main.c, include the header using: #include "../include/category.h"
- Go to Project > Add to Project and add category.c
- Press F9 to Compile and Run

8. Dependencies

Module 2 depends on the following:

Dependency	Reason
stdio.h	Used for printf() and scanf() in all functions
stdlib.h	Used for malloc() and free() for dynamic memory allocation
string.h	Used for string operations such as strcmp() and strcpy()
person.h	Required because Category contains an array of Person structs

9. GitHub Contribution

All work for Module 2 was committed to the team GitHub repository under the branch module2-category. The following commands were used:

```
git checkout -b module2-category
git add include/category.h src/category.c
git commit -m "Module 2: implemented all 7 Category Management functions"
git push origin module2-category
```

10. Conclusion

Module 2 successfully implements a complete Category Management system using a dynamic singly linked list in C. All 7 required functions have been implemented and tested. The module correctly depends on Module 1 through person.h and is ready to be integrated with Module 3 (Priority Management) and Module 5 (User Interface). The code follows the modular architecture specified in the project requirements and has been pushed to the team GitHub repository.

4. Integration with Other Modules

The diagram below shows how types.h and ui.c sit relative to the other modules. Every arrow represents a #include dependency or a function call.

Component	Depends On	Called By
types.h	(none foundation layer)	All headers and .c files
person.h / person.c	types.h	category.c, ui.c
category.h / category.c	types.h, person.h	ui.c, priority.c
priority.h / priority.c	types.h, category.h	ui.c
gift.h / gift.c	types.h, person.h	ui.c

Component	Depends On	Called By
ui.h / ui.c	types.h, person.h, category.h, priority.h, gift.h	main.c
main.c	ui.h, category.h, gift.h	(entry point called by OS)

5. Testing & Validation

5.1 Compilation Test

The first validation step is a clean compilation with no warnings. The Makefile uses `-Wall` and `-Wextra` to enable all warnings, treating them as early indicators of bugs.

5.2 Include Guard Test

To verify include guards work correctly, temporarily add a second `#include "types.h"` in any `.c` file and recompile. If the guard is functioning, the compiler should produce zero 'redefinition' errors. Remove the duplicate after testing.

7. Challenges Encountered & Solutions Applied

Challenge	Root Cause	Solution Applied
<code>scanf()</code> leaving '\n' in buffer causing <code>fgets()</code> to read empty lines	<code>scanf</code> does not consume the newline after reading	Implemented <code>clear_input_buffer()</code> called after every <code>scanf()</code>
Include errors when <code>types.h</code> included in multiple <code>.c</code> files	No include guard struct redefined multiple times	Added <code>#ifndef TYPES_H / #define TYPES_H / #endif</code> wrapper
Infinite loop when user types letters at a numeric menu prompt	<code>scanf("%d")</code> fails silently, bad chars stay in buffer	Implemented <code>read_int()</code> checks <code>scanf</code> return value and drains buffer on failure
Self-referential pointer syntax error in <code>Category</code> struct	Typedef alias not yet complete inside its own struct body	Used <code>'struct Category *next'</code> instead of <code>'Category *next'</code> inside the struct definition
Screen content disappearing before user could read it	<code>system("clear")</code> called immediately at top of next loop iteration	Added <code>'Press Enter to continue...'</code> pause before screen is cleared

8. Conclusion

This report has documented the complete design and implementation of two components central to the WIGMS application: the shared types library (`types.h`) and the User Interface module (Module 5, `ui.h` and `ui.c`).

`types.h` establishes the data foundation that all modules depend on. Its use of include guards, typedef enums, and a self-referential struct demonstrates core C type system concepts in a practical context. Module 5 demonstrates the MVC architectural pattern in C, showing how to cleanly separate user interaction from business logic through a controlled menu loop, safe integer input validation, and disciplined delegation to other modules.

Both components have been successfully integrated and compile cleanly under GCC with `-Wall -Extra`. The identified challenges particularly the `scanf` buffer issue and the include guard requirement were resolved and the solutions have been documented for team knowledge sharing.

Next steps for the team are to complete the remaining modules and conduct full integration testing before the final submission deadline.

3. Module 3; Priority Management (DPR Merge Sort)

3.1 Objectives

The main objectives of Module 3 are:

- Implement the DPR (Divide-Process-Recombine) algorithmic paradigm on the category linked list
- Sort guests within each category by age using insertion sort on the small fixed-size array
- Sort guests by social class priority (VIP first, then Family, then Friend, then Other)
- Apply Merge Sort to the category linked list itself, ordering categories by guest count

3.2 What Is DPR?

DPR stands for Divide – Process – Recombine. It is the academic name given in this project to the Merge Sort algorithm. The algorithm operates in three recursive steps:

Step	Action	Function
DIVIDE	Find the middle of the linked list and split it into two halves	get_middle() using Floyd tortoise & hare
PROCESS	Recursively sort both halves independently	merge_sort_internal() calling itself
RECOMBINE	Merge the two sorted halves into one sorted list	merge() comparing heads one by one

3.3 Why Merge Sort for a Linked List?

Merge Sort is the ideal algorithm for linked lists because it does not require random access by index (which arrays have but linked lists do not). It achieves $O(n \log n)$ time complexity in all cases and is stable, meaning equal elements preserve their original relative order. Bubble sort, used in Module 2 for a simpler sort, has $O(n^2)$ complexity and would be too slow for large guest lists.

3.4 File Structure

File	Role
include/priority.h	Header function declarations and include guard
src/priority.c	Implementation all DPR functions

3.5 Key Functions

3.5.1 get_middle() Floyd's Tortoise & Hare

This function finds the middle node of the linked list in a single pass using two pointers: slow moves one step at a time and fast moves two steps at a time. When fast reaches the end, slow is at the midpoint. This avoids counting the list length first, saving an extra traversal.

3.5.2 merge() The Recombine Step

The merge function receives two already-sorted sub-lists and produces one combined sorted list. It compares the heads of each sub-list and attaches the smaller one to the result, then recurses with the remaining nodes. When one sub-list is exhausted, the remaining nodes of the other are appended directly.

3.5.3 `prioritize_by_age()` and `prioritize_by_social_class()`

These functions sort the `guests[]` array inside each Category node. Because the array holds at most 4 elements, insertion sort is used it is faster than merge sort for very small arrays due to lower overhead. The `class_rank()` helper maps social class strings to integer priority values: VIP=1, Family=2, Friend=3, Other=4.

3.6 Step-by-Step Implementation Procedure

STEP 1 Create `priority.h`

Create `include/priority.h`. Add the include guard (`#ifndef PRIORITY_H / #define PRIORITY_H / #endif`). Declare the three public functions: `priority_sort_by_age()`, `priority_sort_by_class()`, `priority_merge_sort_categories()`. Mark internal helpers as static in `priority.c` so they are invisible outside the file.

STEP 2 Implement `get_middle()`

In `src/priority.c`, implement `get_middle()` with the slow/fast pointer technique described above. Test with a manually built 5-node list and verify slow lands on node 3.

STEP 3 Implement `merge()`

Implement the `merge()` function with the `by_age` flag controlling which field is used for comparison. Test by creating two pre-sorted 3-node sub-lists and verifying the merged result is in order.

STEP 4 Implement `merge_sort_internal()`

Implement the recursive DPR function: find middle, set `mid->next = NULL` to cut the list, recursively sort both halves, merge results. Test with a fully unsorted 6-node list.

STEP 5 Implement insertion sort for guests

Implement `priority_sort_by_age()` and `priority_sort_by_class()` using insertion sort on the fixed `guests[4]` array inside each node.

STEP 6 Commit to GitHub

`git add include/priority.h src/priority.c && git commit -m 'Module 3: DPR Merge Sort complete' && git push origin module3-priority`

3.7 GitHub Contribution

```
git checkout -b module3-priority
git add include/priority.h src/priority.c
git commit -m "Module 3: implemented DPR Merge Sort sort by age, class, guest count"
git push origin module3-priority
```

```
1  #ifndef PRIORITY_H
2  #define PRIORITY_H
3
4  #include "types.h"
5
6  /* =====
7   MODULE 3 - Priority Management (DPR / Merge Sort)
8   ===== */
9
10 /* Console versions */
11 void prioritize_by_age(Category *tete);
12 void prioritize_by_social_class(Category *tete);
13 void merge_sort_categories(Category **tete);
14
15 /* GUI-friendly wrappers (same behavior, different names) */
16 void priority_sort_by_age(Category *tete);
17 void priority_sort_by_class(Category *tete);
18 void priority_merge_sort_categories(Category **tete);
19
20 #endif /* PRIORITY_H */
21 |
```



```

1  #include <stdio.h>
2  #include <string.h>
3
4  #include "priority.h"
5
6  static Category* get_middle(Category *head) {
7      if (!head) return NULL;
8
9      Category *slow = head;
10     Category *fast = head->next;
11
12     while (fast != NULL && fast->next != NULL) {
13         slow = slow->next;
14         fast = fast->next->next;
15     }
16     return slow;
17 }
18
19 static Category* merge(Category *left, Category *right, int by_age) {
20     if (left == NULL) return right;
21     if (right == NULL) return left;
22
23     int left_val;
24     int right_val;
25
26     if (by_age) {
27         left_val = (left->guest_count > 0) ? left->guests[0].age : 0;
28         right_val = (right->guest_count > 0) ? right->guests[0].age : 0;
29     } else {
30         left_val = left->guest_count;
31         right_val = right->guest_count;
32     }
33
34     if (left_val <= right_val) {
35         left->next = merge(left->next, right, by_age);
36         return left;
37     } else {
38         right->next = merge(left, right->next, by_age);
39         return right;
40     }
41 }
42
43 static Category* merge_sort_internal(Category *head, int by_age) {
44     if (!head || !head->next) return head;
45
46     Category *mid = get_middle(head);
47     Category *right = mid->next;
48     mid->next = NULL;
49
50     Category *left_sorted = merge_sort_internal(head, by_age);
51     Category *right_sorted = merge_sort_internal(right, by_age);
52
53     return merge(left_sorted, right_sorted, by_age);
54 }
55
56 void prioritize_by_age(Category *tete) {
57     Category *curr = tete;
58     while (curr != NULL) {
59         for (int i = 1; i < curr->guest_count; i++) {
60             Person key = curr->guests[i];
61             int j = i - 1;
62             while (j >= 0 && curr->guests[j].age > key.age) {
63                 curr->guests[j + 1] = curr->guests[j];
64                 j--;
65             }
66             curr->guests[j + 1] = key;
67         }
68         curr = curr->next;
69     }
70     printf("  Guests within each category sorted by age (ascending).\n");
71 }
72
73 static int class_rank(const char *cls) {
74     if (!cls) return 4;
75     if (strcmp(cls, "VIP") == 0) return 1;
76     if (strcmp(cls, "Family") == 0) return 2;
77     if (strcmp(cls, "Friend") == 0) return 3;
78     return 4;
79 }
80
81 void prioritize_by_social_class(Category *tete) {
82     Category *curr = tete;
83     while (curr != NULL) {
84         for (int i = 1; i < curr->guest_count; i++) {
85             Person key = curr->guests[i];
86             int j = i - 1;
87             while (j >= 0 &&
88                 class_rank(curr->guests[j].social_class) >
89                 class_rank(key.social_class)) {
90                 curr->guests[j + 1] = curr->guests[j];
91                 j--;
92             }
93             curr->guests[j + 1] = key;
94         }
95         curr = curr->next;
96     }
97     printf("  Guests sorted by social class (VIP first).\n");
98 }
99
100 void merge_sort_categories(Category **tete) {
101     if (!*tete) return;
102     *tete = merge_sort_internal(*tete, 0 /* by guest count */);
103     printf("  Categories sorted by guest count using Merge Sort (DPR).\n");
104 }
105
106 void priority_sort_by_age(Category *tete) {
107     prioritize_by_age(tete);
108 }
109
110 void priority_sort_by_class(Category *tete) {
111     prioritize_by_social_class(tete);
112 }
113
114 void priority_merge_sort_categories(Category **tete) {
115     merge_sort_categories(tete);
116 }
117

```

4. Module 4: Gift Management

4.1 Objectives

- Register gifts linked to specific guests by `guest_id`
- Display all gifts with their monetary value in FCFA
- Update and delete gifts by `gift_id`
- Calculate the total gift value for the entire wedding
- Save gifts to a binary file on disk so data survives program restarts
- Load gifts automatically from the binary file at startup

4.2 Data Structure

Module 4 uses a fixed-size array of Gift structs rather than a linked list. This choice is appropriate because gifts are frequently accessed by index, the maximum count is bounded (`MAX_GIFTS = 200`), and the flat array structure makes binary file I/O trivially simple one `fwrite` call writes all gifts at once.

```
typedef struct {
    int gift_id;
    char name[100];
    float value; /* in FCFA */
    int guest_id; /* foreign key links to a guest */
} Gift;

#define MAX_GIFTS 200
```

4.3 File Persistence, Binary I/O

The most technically distinctive feature of Module 4 is binary file persistence. When the program exits, all RAM is cleared. To keep gift data between sessions, the gifts array is written to a `.dat` file using `fwrite()` and read back with `fread()` at the next startup.

Function	Mode	What It Does
<code>gift_save()</code>	<code>fopen("wb")</code> write binary	Writes the integer count as a header, then writes all Gift structs in one <code>fwrite()</code> call
<code>gift_load()</code>	<code>fopen("rb")</code> read binary	Reads the count header first, then reads exactly that many Gift structs back into the array

4.4 Array Delete Shift Pattern

Because arrays do not support true deletion, removing a gift at index `i` requires shifting every element after it one position to the left, then decrementing the count. The last element is effectively orphaned beyond the active range and will be overwritten by the next insertion.

4.5 Step-by-Step Implementation Procedure

STEP 1 Create `gift.h`

Write `include/gift.h` with the Gift struct, `MAX_GIFTS` macro, `extern gift_id_counter`, and all function declarations.

STEP 2 Implement CRUD functions

In src/gift.c implement gift_register() (array append with bounds check), gift_delete() (array shift), gift_update() (linear search then modify), and gift_total_value() (accumulator loop).

STEP 3 Implement file I/O

Implement gift_save() and gift_load() exactly as shown above. Test by saving 3 gifts, closing the program in Dev-C++, running again, and verifying the 3 gifts are reloaded.

STEP 4 Integrate in main.c

Call gift_load() at program startup before the menu loop. Call gift_save() before program exit.

STEP 5 Test persistence

Add 5 gifts, close the program, reopen, verify all 5 reload correctly and gift IDs continue from where they left off.

STEP 6 Commit to GitHub

git add include/gift.h src/gift.c && git commit -m 'Module 4: Gift CRUD + binary file persistence' && git push origin module4-gifts

```
priority.c  priority.h  gift.h  gift.c
1  #ifndef GIFT_H
2  #define GIFT_H
3
4  #include "types.h"
5
6  /* =====
7  MODULE 4 - Gift Management (Array + File Persistence)
8  ===== */
9
10 /* Console-interactive versions */
11 void register_gift(Gift gifts[], int *count);
12 void display_gifts(Gift gifts[], int count);
13 void update_gifts(Gift gifts[], int count, int gift_id);
14 void delete_gift(Gift gifts[], int *count, int gift_id);
15 float total_gift_value(Gift gifts[], int count);
16 void save_gifts(Gift gifts[], int count, const char *filename);
17 void load_gifts(Gift gifts[], int *count, const char *filename);
18
19 extern int gift_id_counter;
20
21 /* =====
22 GUI-friendly wrappers (data-driven)
23 ===== */
24 void gift_register(Gift gifts[], int *count,
25                  const char *name, float value, int guest_id);
26 void gift_display(Gift gifts[], int count); /* optional, not required */
27 void gift_update(Gift gifts[], int count, int gift_id,
28                 const char *name, float value, int guest_id);
29 void gift_delete(Gift gifts[], int *count, int gift_id);
30 float gift_total_value(Gift gifts[], int count);
31 void gift_save(Gift gifts[], int count, const char *filename);
32 void gift_load(Gift gifts[], int *count, const char *filename);
33
34 #endif /* GIFT_H */
35
```

C: > Users > hp > Desktop > PROJECT 1 > Wedding Test > src > C gift.c > display_gifts(Gift [], int)

```

1  #include <stdio.h> Use Ctrl + I to generate code
2  #include <string.h>
3  #include <errno.h>
4
5  #ifdef _WIN32
6  | #include <direct.h>
7  #else
8  | #include <sys/stat.h>
9  #endif
10
11 #include "gift.h"
12 #include "person.h" /* for clear_input_buffer() */
13
14 int gift_id_counter = 1;
15
16 Windsurf: Refactor | Explain | Generate Function Comment | X
17 static void ensure_data_dir(void) {
18     /* Best-effort: ignore errors if it already exists */
19     _mkdir("data");
20 #else
21     mkdir("data", 0755);
22 #endif
23 }
24
25 /* -----
26    register_gift() - console-interactive
27    ----- */
28 Windsurf: Refactor | Explain | X
29 void register_gift(Gift gifts[], int *count) {
30     if (!gifts || !count) return;
31     if (*count >= MAX_GIFTS) {
32         printf(" Gift registry is full (%d gifts).\n", MAX_GIFTS);
33         return;
34     }
35
36     Gift g;
37     g.gift_id = gift_id_counter++;
38
39     printf(" Gift name      : ");
40     fgets(g.name, sizeof(g.name), stdin);
41     g.name[strcspn(g.name, "\n")] = '\0';
42
43     printf(" Gift value (FCFA): ");
44     scanf("%f", &g.value);
45     clear_input_buffer();
46
47     printf(" Guest ID who gave this gift: ");
48     scanf("%d", &g.guest_id);
49     clear_input_buffer();
50
51     gifts[*count] = g;
52     (*count)++;
53
54     printf(" Gift '%s' registered (ID: %d).\n", g.name, g.gift_id);
55 }
56
57 Windsurf: Refactor | Explain | Generate Function Comment | X
58 void display_gifts(Gift gifts[], int count) {
59     if (!gifts || count <= 0) {
60         printf(" No gifts registered yet.\n");
61         return;
62     }
63
64     printf("\n %5s %-25s %-12s %-10s\n",
65           "ID", "Gift Name", "Value(FCFA)", "Guest ID");
66     printf(" %5s %-25s %-12s %-10s\n",
67           "----", "-----", "-----", "-----");
68
69     float total = 0.0f;
70     for (int i = 0; i < count; i++) {
71         printf(" %5d %-25s %-12.0f %-10d\n",
72               gifts[i].gift_id,

```


C: > Users > hp > Desktop > PROJECT 1 > Wedding Test > src > C gift.c > ...

```

163 void load_gifts(Gift gifts[], int *count, const char *filename) {
164     if (read > 0) {
165         gift_id_counter = gifts[*count - 1].gift_id + 1;
166     } else {
167         gift_id_counter = 1;
168     }
169 }
170
171 /* =====
172 GUI-friendly wrappers expected by ui_gtk.c
173 ===== */
174
175 Windsurf: Refactor | Explain | Generate Function Comment | X
176 void gift_register(Gift gifts[], int *count,
177     const char *name, float value, int guest_id) {
178     if (!gifts || !count) return;
179     if (*count >= MAX_GIFTS) return;
180
181     Gift g;
182     g.gift_id = gift_id_counter++;
183     strncpy(g.name, name ? name : "Gift", sizeof(g.name) - 1);
184     g.name[sizeof(g.name) - 1] = '\0';
185     g.value = value;
186     g.guest_id = guest_id;
187
188     gifts[*count] = g;
189     (*count)++;
190 }
191
192 Windsurf: Refactor | Explain | Generate Function Comment | X
193 void gift_display(Gift gifts[], int count) {
194     display_gifts(gifts, count);
195 }
196
197 Windsurf: Refactor | Explain | Generate Function Comment | X
198 void gift_update(Gift gifts[], int count, int gift_id,
199     const char *name, float value, int guest_id) {
200     if (!gifts) return;
201     int idx = -1;
202     for (int i = 0; i < count; i++) {
203         if (gifts[i].gift_id == gift_id) {
204             idx = i;
205             break;
206         }
207     }
208     if (idx == -1) return;
209
210     if (name && name[0]) {
211         strncpy(gifts[idx].name, name, sizeof(gifts[idx].name) - 1);
212         gifts[idx].name[sizeof(gifts[idx].name) - 1] = '\0';
213     }
214     gifts[idx].value = value;
215     gifts[idx].guest_id = guest_id;
216 }
217
218 Windsurf: Refactor | Explain | Generate Function Comment | X
219 void gift_delete(Gift gifts[], int *count, int gift_id) {
220     delete_gift(gifts, count, gift_id);
221 }
222
223 Windsurf: Refactor | Explain | Generate Function Comment | X
224 float gift_total_value(Gift gifts[], int count) {
225     return total_gift_value(gifts, count);
226 }
227
228 Windsurf: Refactor | Explain | Generate Function Comment | X
229 void gift_save(Gift gifts[], int count, const char *filename) {
230     save_gifts(gifts, count, filename);
231 }
232
233 Windsurf: Refactor | Explain | Generate Function Comment | X
234 void gift_load(Gift gifts[], int *count, const char *filename) {
235     load_gifts(gifts, count, filename);
236 }

```

```

163 void load_gifts(Gift gifts[], int *count, const char *filename) {
164     if (read > 0) {
165         gift_id_counter = gifts[*count - 1].gift_id + 1;
166     } else {
167         gift_id_counter = 1;
168     }
169 }
170
171 /* =====
172 GUI-friendly wrappers expected by ui_gtk.c
173 ===== */
174
175 Windsurf: Refactor | Explain | Generate Function Comment | X
176 void gift_register(Gift gifts[], int *count,
177     const char *name, float value, int guest_id) {
178     if (!gifts || !count) return;
179     if (*count >= MAX_GIFTS) return;
180
181     Gift g;
182     g.gift_id = gift_id_counter++;
183     strncpy(g.name, name ? name : "Gift", sizeof(g.name) - 1);
184     g.name[sizeof(g.name) - 1] = '\0';
185     g.value = value;
186     g.guest_id = guest_id;
187
188     gifts[*count] = g;
189     (*count)++;
190 }
191
192 Windsurf: Refactor | Explain | Generate Function Comment | X
193 void gift_display(Gift gifts[], int count) {
194     display_gifts(gifts, count);
195 }
196
197 Windsurf: Refactor | Explain | Generate Function Comment | X
198 void gift_update(Gift gifts[], int count, int gift_id,
199     const char *name, float value, int guest_id) {
200     if (!gifts) return;
201     int idx = -1;
202     for (int i = 0; i < count; i++) {
203         if (gifts[i].gift_id == gift_id) {
204             idx = i;
205             break;
206         }
207     }
208     if (idx == -1) return;
209
210     if (name && name[0]) {
211         strncpy(gifts[idx].name, name, sizeof(gifts[idx].name) - 1);
212         gifts[idx].name[sizeof(gifts[idx].name) - 1] = '\0';
213     }
214     gifts[idx].value = value;
215     gifts[idx].guest_id = guest_id;
216 }
217
218 Windsurf: Refactor | Explain | Generate Function Comment | X
219 void gift_delete(Gift gifts[], int *count, int gift_id) {
220     delete_gift(gifts, count, gift_id);
221 }
222
223 Windsurf: Refactor | Explain | Generate Function Comment | X
224 float gift_total_value(Gift gifts[], int count) {
225     return total_gift_value(gifts, count);
226 }
227
228 Windsurf: Refactor | Explain | Generate Function Comment | X
229 void gift_save(Gift gifts[], int count, const char *filename) {
230     save_gifts(gifts, count, filename);
231 }
232
233 Windsurf: Refactor | Explain | Generate Function Comment | X
234 void gift_load(Gift gifts[], int *count, const char *filename) {
235     load_gifts(gifts, count, filename);
236 }

```

5. Module 5: Graphical User Interface (GTK+ 3.0)

5.1 Objectives

Module 5 replaces the original console menu system with a complete graphical user interface built using the GTK+ 3.0 library. The interface provides a professional windowed application with five tabs, interactive data tables, forms, modal dialogs, CSS theming, and a live statistics dashboard. It acts as the Controller layer of the MVC architecture it contains no business logic, only routing user actions to the correct module functions.

5.2 What Is GTK?

GTK (GIMP Toolkit) is a professional open-source C library for building graphical user interfaces with real windows, buttons, forms, menus, and dialogs. It is the same library used by the GNOME desktop environment and applications like GIMP and Inkscape. It was chosen for this project because it integrates directly with C, requires no additional language, and is available on Linux via a simple apt install command.

```
Install command: sudo apt install libgtk-3-dev pkg-config
Compile flag: $(shell pkg-config --cflags --libs gtk+-3.0)
```

5.3 GTK Architecture The Widget Tree

GTK builds a GUI as a tree of widgets. Every element visible on screen a window, a button, a text field, a table is a widget. Widgets are nested inside containers (boxes, grids, notebooks). The entire tree is made visible with one call to `gtk_widget_show_all()`.

```
GtkWindow (main application window)
├── GtkNotebook (5 tabs)
│   ├── Tab 1: Categories → GtkBox → GtkTreeView + GtkGrid form
│   ├── Tab 2: Persons → GtkBox → GtkTreeView + GtkGrid form
│   ├── Tab 3: Priority → GtkBox → buttons + GtkTreeView
│   ├── Tab 4: Gifts → GtkBox → GtkTreeView + GtkGrid form
│   └── Tab 5: Dashboard → GtkBox → stat card GtkFrames
```

5.4 Signal-Callback Event Model

GTK is event-driven. After calling `gtk_main()`, the program waits in an event loop. When the user clicks a button or selects a row, GTK fires a signal. The `g_signal_connect()` function wires a signal to a callback function written in C.

5.5 Key GTK Widgets Used

Widget	Purpose in WIGMS
GtkWindow	Top-level application window root of the widget tree
GtkHeaderBar	Modern title bar with embedded subtitle
GtkNotebook	5-tab panel container
GtkBox	Linear container for vertical and horizontal layouts

Widget	Purpose in WIGMS
GtkGrid	2-D layout for all data entry forms
GtkTreeView + GtkListStore	Spreadsheet-style data table with live model
GtkEntry	Single-line text input names, codes, plates
GtkSpinButton	Numeric up/down spinner age, value, count
GtkComboBoxText	Drop-down list Side, social class, vehicle type
GtkButton	Clickable button Add, Update, Delete, Sort, Save
GtkLabel	Static and dynamic text titles, status, totals
GtkMessageDialog	Pre-built modal dialog for info, errors, and confirmations
GtkCssProvider	Loads CSS rules for the navy/orange colour theme
GtkScrolledWindow	Adds scroll bars around TreeView tables
GtkFrame	Labelled border box used for Dashboard stat cards

5.6 CSS Theming

GTK 3 supports CSS styling identical in syntax to web CSS. A CSS string is loaded at application startup and applied globally to the screen via GtkCssProvider. This gives all buttons, labels, trees, and stat cards a consistent professional colour scheme using the project's navy and orange palette.

5.7 AppState Shared Application Data

All GTK tabs share a single AppState struct declared in ui_gtk.h. A pointer to this struct is passed as user_data to every callback. This ensures that changes made on one tab (such as adding a category) are immediately visible on all other tabs.

5.8 Step-by-Step Implementation Procedure

STEP 1 Install GTK

Run: `sudo apt install libgtk-3-dev pkg-config`. Verify: `pkg-config --modversion gtk+-3.0` should print 3.x.x

Using GTK from MSYS2 packages

Installation

The [MSYS2](#) project provides a UNIX-like development environment for Windows. It provides packages for many software applications and libraries, including the GTK stack. If you prefer developing using Visual Studio, you should use `gvsbuild` instead.

In MSYS2 packages are installed using the [pacman package manager](#).

Note: in the following steps, we will assume you're using a [64-bit Windows](#). Therefore, the package names include the `x86_64` architecture identifier. If you're using a 32-bit Windows, please adapt the instructions below using the `i686` architecture identifier.

Step 1.: Download the [MSYS2 installer](#) that matches your platform and follow the installation instructions.

Step 2.: Install GTK4 and its dependencies. Open a MSYS2 shell, and run:

```
pacman -S mingw-w64-ucrt-x86_64-gtk4
```

If you want to develop with GTK3, run:

```
pacman -S mingw-w64-ucrt-x86_64-gtk3
```

Step 3. (optional): If you want to develop a GTK application in C, C++, Fortran, etc, you'll need a compiler like GCC and its toolchain:

```
pacman -S mingw-w64-ucrt-x86_64-toolchain base-devel
```

If you want to develop a GTK application in Python, you need to install the Python bindings:

```
pacman -S mingw-w64-ucrt-x86_64-python-gobject
```

If you want to develop a GTK application in [Vala](#), you will need to install the Vala package:

```
pacman -S mingw-w64-ucrt-x86_64-vala
```

Building and distributing your application



Once you have installed the GTK as above, you should have little problem compiling a GTK app. In order to run it successfully, you will also need a GTK theme. There is some old builtin support for a Windows theme in GTK, but that makes your app look like a Windows 7 app. It is better to get a Windows 10 theme, for instance the [Windows 10 Transformation Pack](#).

STEP 2 Update Makefile

Add `$(shell pkg-config --cflags gtk+-3.0)` to CFLAGS and `$(shell pkg-config --libs gtk+-3.0)` to LIBS in the Makefile.

The screenshot shows the MSYS2 website's 'Getting Started' page on the left and a terminal window on the right. The website lists steps for installation, including downloading the installer, running it, and specifying an installation folder. The terminal window shows the command `$ pacman -S mingw-w64-ucrt-x86_64-gcc` and the output of the package manager, which lists various packages to be installed, including `mingw-w64-ucrt-x86_64-binutils`, `mingw-w64-ucrt-x86_64-crt`, `mingw-w64-ucrt-x86_64-gcc`, `mingw-w64-ucrt-x86_64-gmp`, `mingw-w64-ucrt-x86_64-headers`, `mingw-w64-ucrt-x86_64-isl`, `mingw-w64-ucrt-x86_64-libiconv`, `mingw-w64-ucrt-x86_64-libwinpthread`, `mingw-w64-ucrt-x86_64-mpfr`, `mingw-w64-ucrt-x86_64-mpc`, `mingw-w64-ucrt-x86_64-tzdata`, `mingw-w64-ucrt-x86_64-windows-default-manifest`, `mingw-w64-ucrt-x86_64-winpthreads`, `mingw-w64-ucrt-x86_64-zlib`, and `mingw-w64-ucrt-x86_64-zstd`. The terminal also shows the progress of the installation, including the total download size (69.89 MiB) and total installed size (550.10 MiB).

The screenshot shows a terminal window with the output of the `pacman` command. The output displays the progress of installing various packages, including `mingw-w64-ucrt-x86_64-binutils`, `mingw-w64-ucrt-x86_64-crt`, `mingw-w64-ucrt-x86_64-gcc`, `mingw-w64-ucrt-x86_64-gmp`, `mingw-w64-ucrt-x86_64-headers`, `mingw-w64-ucrt-x86_64-isl`, `mingw-w64-ucrt-x86_64-libiconv`, `mingw-w64-ucrt-x86_64-libwinpthread`, `mingw-w64-ucrt-x86_64-mpfr`, `mingw-w64-ucrt-x86_64-mpc`, `mingw-w64-ucrt-x86_64-tzdata`, `mingw-w64-ucrt-x86_64-windows-default-manifest`, `mingw-w64-ucrt-x86_64-winpthreads`, `mingw-w64-ucrt-x86_64-zlib`, and `mingw-w64-ucrt-x86_64-zstd`. The progress is shown as a percentage (00%) and a bar chart. The terminal also shows the command `pacman -S mingw-w64-ucrt-x86_64-gcc` and the output of the package manager, which lists various packages to be installed, including `mingw-w64-ucrt-x86_64-binutils`, `mingw-w64-ucrt-x86_64-crt`, `mingw-w64-ucrt-x86_64-gcc`, `mingw-w64-ucrt-x86_64-gmp`, `mingw-w64-ucrt-x86_64-headers`, `mingw-w64-ucrt-x86_64-isl`, `mingw-w64-ucrt-x86_64-libiconv`, `mingw-w64-ucrt-x86_64-libwinpthread`, `mingw-w64-ucrt-x86_64-mpfr`, `mingw-w64-ucrt-x86_64-mpc`, `mingw-w64-ucrt-x86_64-tzdata`, `mingw-w64-ucrt-x86_64-windows-default-manifest`, `mingw-w64-ucrt-x86_64-winpthreads`, `mingw-w64-ucrt-x86_64-zlib`, and `mingw-w64-ucrt-x86_64-zstd`.

```
M ~
$ pacman -S --needed \
> mingw-w64-x86_64-toolchain \
> mingw-w64-x86_64-pkg-config \
> mingw-w64-x86_64-gtk3 \
> make
:: There are 13 members in group mingw-w64-x86_64-toolchain:
:: Repository mingw64
 1) mingw-w64-x86_64-binutils 2) mingw-w64-x86_64-crt 3) mingw-w64-x86_64-gcc
 4) mingw-w64-x86_64-gdb 5) mingw-w64-x86_64-gdb-multiarch 6) mingw-w64-x86_64-headers
 7) mingw-w64-x86_64-libmangle 8) mingw-w64-x86_64-libwinpthread 9) mingw-w64-x86_64-make
10) mingw-w64-x86_64-pkgconf 11) mingw-w64-x86_64-tools 12) mingw-w64-x86_64-winpthread
13) mingw-w64-x86_64-winstorecompat

Enter a selection (default=all):
warning: make-4.4.1-2 is up to date -- skipping
resolving dependencies...
looking for conflicting packages...
warning: removing 'mingw-w64-x86_64-pkg-config-0.29.2-6' from target list because it conflicts with 'mi
ngw-w64-x86_64-pkgconf-1~2.5.1-1'
warning: dependency cycle detected:
warning: mingw-w64-x86_64-libwebp will be installed before its mingw-w64-x86_64-libtiff dependency
warning: dependency cycle detected:
warning: mingw-w64-x86_64-harfbuzz will be installed before its mingw-w64-x86_64-freetype dependency

Packages (77) mingw-w64-x86_64-adwaita-icon-theme-50.0-1
mingw-w64-x86_64-adwaita-icon-theme-legacy-46.2-1 mingw-w64-x86_64-atk-2.58.4-1
mingw-w64-x86_64-brotli-1.2.0-1 mingw-w64-x86_64-bzip2-1.0.8-3
mingw-w64-x86_64-cairo-1.18.4-4 mingw-w64-x86_64-egl-headers-1.5.r284.3ae2b7c-1
mingw-w64-x86_64-expat-2.7.5-2 mingw-w64-x86_64-fontconfig-2.17.1-1
mingw-w64-x86_64-freetype-2.14.3-1 mingw-w64-x86_64-fribidi-1.0.16-1
mingw-w64-x86_64-gcc-libs-15.2.0-13 mingw-w64-x86_64-gdk-pixbuf2-2.44.5-1
mingw-w64-x86_64-gettext-runtime-1.0-1 mingw-w64-x86_64-giflib-5.2.2-1
mingw-w64-x86_64-gles-headers-3.2.r1065.7fc154c-1 mingw-w64-x86_64-glib2-2.88.0-1
mingw-w64-x86_64-gmp-6.3.0-2 mingw-w64-x86_64-graphite2-1.3.14-3
mingw-w64-x86_64-gtk-update-icon-cache-3.24.52-1 mingw-w64-x86_64-harfbuzz-13.2.1-1
mingw-w64-x86_64-hicolor-icon-theme-0.18-1 mingw-w64-x86_64-isl-0.27-1
mingw-w64-x86_64-jbigkit-2.1-5 mingw-w64-x86_64-json-glib-1.10.8-2
mingw-w64-x86_64-lerc-4.1.0-1 mingw-w64-x86_64-libb2-0.98.1-3
mingw-w64-x86_64-libdatatr-0.2.14-1 mingw-w64-x86_64-libdeflate-1.25-1
mingw-w64-x86_64-libepoxy-1.5.10-7 mingw-w64-x86_64-libffi-3.5.2-1
mingw-w64-x86_64-libiconv-1.19-1 mingw-w64-x86_64-libjpeg-turbo-3.1.4.1-1
mingw-w64-x86_64-libpng-1.6.56-1 mingw-w64-x86_64-librsvg-2.62.0-1
mingw-w64-x86_64-libthai-0.1.30-1 mingw-w64-x86_64-libtiff-4.7.1-1
mingw-w64-x86_64-libwebp-1.6.0-1 mingw-w64-x86_64-libxml2-2.15.2-1
mingw-w64-x86_64-lzo2-2.10-3 mingw-w64-x86_64-mpc-1.4.0-1
mingw-w64-x86_64-mpdecimal-4.0.1-1 mingw-w64-x86_64-mpfr-4.2.2-1
mingw-w64-x86_64-ncurses-6.6-3 mingw-w64-x86_64-openssl-3.6.1-3
mingw-w64-x86_64-pango-1.56.4-3 mingw-w64-x86_64-pcre2-10.47-1
mingw-w64-x86_64-pixman-0.46.4-1 mingw-w64-x86_64-python-3.14.3-1
mingw-w64-x86_64-python-packaging-26.0-2 mingw-w64-x86_64-readline-8.3.003-1
mingw-w64-x86_64-shared-mime-info-2.4-4 mingw-w64-x86_64-sqlite3-3.52.0-1
mingw-w64-x86_64-tcl-8.6.17-1 mingw-w64-x86_64-termcap-1.3.1-7
mingw-w64-x86_64-tk-8.6.17-2 mingw-w64-x86_64-tzdata-2026a-1
mingw-w64-x86_64-windows-default-manifest-6.4-4 mingw-w64-x86_64-wineditline-2.208-1
mingw-w64-x86_64-xxhash-0.8.3-1 mingw-w64-x86_64-xz-5.8.3-1
mingw-w64-x86_64-zlib-1.3.2-2 mingw-w64-x86_64-zstd-1.5.7-1
mingw-w64-x86_64-binutils-2.46-2 mingw-w64-x86_64-crt-14.0.0.r1.gbd663f0e1-1
mingw-w64-x86_64-gcc-15.2.0-13 mingw-w64-x86_64-gdb-17.1-3
```

STEP 3 Create ui_gtk.h

Write the AppState struct, declare ui_gtk_run() as the entry point, and declare all shared helper functions (dialogs, CSS).

STEP 4 Implement ui_apply_css()

Write the CSS string as a C string literal and load it with gtk_css_provider_load_from_data(). Apply globally with gtk_style_context_add_provider_for_screen().

STEP 5 Build the main window

In ui_gtk_run(): call gtk_init(), create GtkWindow, attach GtkHeaderBar, create GtkNotebook, call each build_*_tab() function, append each returned widget as a tab page.

STEP 6 Build each tab

Implement `build_category_tab()`, `build_person_tab()`, `build_priority_tab()`, `build_gift_tab()`, and `build_stats_tab()`. Each returns a `GtkWidget*` root container.

STEP 7 Connect destroy signal

`g_signal_connect(win, 'destroy', G_CALLBACK(on_main_destroy), state)` this auto-saves gifts and frees the category list when the window is closed.

STEP 8 Compile and test

Run `make`. Run `./wigms`. Verify all 5 tabs open, buttons respond, and data displays correctly in the TreeView tables.

```
ui_gtk.h
1  #ifndef UI_GTK_H
2  #define UI_GTK_H
3
4  #include <gtk/gtk.h>
5  #include "types.h"
6  #include "parking.h"
7  #include "seating.h"
8  #include "schedule.h"
9
10 /* =====
11  MODULE 5 - GTK Graphical User Interface
12  ui_gtk.h : public declarations
13  ===== */
14
15 typedef struct {
16     Category *cat_head; /* head of category linked list */
17     Gift gifts[MAX_GIFTS]; /* flat gift array */
18     int gift_count; /* number of active gifts */
19
20     /* Modules 6-8 */
21     ParkingLot parking;
22     DiningHall dining;
23     WeddingSchedule schedule;
24 } AppState;
25
26 /* Entry point for the GTK GUI */
27 void ui_gtk_run(int *argc, char ***argv, AppState *state);
28
29 /* Shared dialog helpers */
30 void ui_apply_css(void);
31 void ui_show_info_dialog(GtkWindow *parent, const char *title, const char *msg);
32 void ui_show_error_dialog(GtkWindow *parent, const char *title, const char *msg);
33 gboolean ui_confirm_dialog(GtkWindow *parent, const char *question);
34
35 #endif /* UI_GTK_H */
36
```



```

1  #include <gtk/gtk.h>
2  #include <string.h>
3  #include <stdlib.h>
4  #include <time.h>
5
6  #include "ui_gtk.h"
7  #include "category.h"
8  #include "person.h"
9  #include "priority.h"
10 #include "gift.h"
11 #include "parking.h"
12 #include "seating.h"
13 #include "schedule.h"
14
15 #define GIFTS_FILE "data/gifts.dat"
16 #define PARKING_FILE "data/parking.dat"
17 #define SEATING_FILE "data/seating.dat"
18 #define SCHEDULE_FILE "data/schedule.dat"
19 #define APP_TITLE "WIGMS - Wedding Invitation & Gift Management"
20 #define APP_W 1100
21 #define APP_H 720
22
23 /* =====
24  Per-tab contexts
25  ===== */
26
27 typedef struct {
28     AppState      *state;
29     GtkWidget     *main_win;
30     GtkListStore  *store;
31     GtkTreeView   *tree;
32     GtkEntry      *entry_code;
33     GtkSpinButton *spin_guests;
34     GtkEntry      *g_name[4];
35     GtkSpinButton *g_age[4];
36     GtkEntry      *g_class[4];
37     GtkComboBoxText *g_side[4];
38     int           selected_id; /* -1 = none */
39 } CatCtx;
40
41 typedef struct {
42     AppState      *state;
43     GtkWidget     *main_win;
44     GtkListStore  *store;
45     GtkTreeView   *tree;
46     GtkEntry      *entry_name;
47     GtkSpinButton *spin_age;
48     GtkEntry      *entry_class;
49     GtkComboBoxText *combo_side;
50     int           selected_cat_id;
51     int           selected_guest_idx;
52 } PersonCtx;
53
54 typedef struct {
55     AppState      *state;
56     GtkWidget     *main_win;
57     GtkListStore  *store;
58     GtkTreeView   *tree;
59     GtkLabel      *lbl_status;
60 } PrioCtx;
61
62 typedef struct {
63     AppState      *state;
64     GtkWidget     *main_win;
65     GtkListStore  *store;
66     GtkTreeView   *tree;
67     GtkEntry      *entry_name;
68     GtkSpinButton *spin_value;
69     GtkSpinButton *spin_guest;
70     GtkLabel      *lbl_total;
71     int           selected_id;
72 } GiftCtx;
73

```

```

73
74 typedef struct {
75     AppState *state;
76     GtkLabel *lbl_cats;
77     GtkLabel *lbl_guests;
78     GtkLabel *lbl_gifts;
79     GtkLabel *lbl_total;
80 } StatsCtx;
81
82 typedef struct {
83     AppState *state;
84     GtkWidget *main_win;
85     GtkListStore *store;
86     GtkTreeView *tree;
87     GtkComboBoxText *combo_zone_filter;
88     GtkSpinButton *spin_guest;
89     GtkEntry *entry_plate;
90     GtkComboBoxText *combo_vehicle;
91     GtkComboBoxText *combo_preferred_zone;
92     GtkLabel *lbl_stats;
93     int selected_spot_id;
94 } ParkCtx;
95
96 typedef struct {
97     AppState *state;
98     GtkWidget *main_win;
99     GtkListStore *store;
100     GtkTreeView *tree;
101     GtkEntry *entry_table_name;
102     GtkSpinButton *spin_table_capacity;
103     GtkComboBoxText *combo_table_type;
104     GtkSpinButton *spin_assign_table;
105     GtkSpinButton *spin_assign_guest;
106     GtkEntry *entry_assign_name;
107     GtkComboBoxText *combo_meal;
108     GtkComboBoxText *combo_diet;
109     GtkSpinButton *spin_move_guest;
110     GtkSpinButton *spin_move_table;
111     GtkSpinButton *spin_rsvp_guest;
112     GtkComboBoxText *combo_rsvp;
113     GtkLabel *lbl_catering;
114 } SeatCtx;
115
116 typedef struct {
117     AppState *state;
118     GtkWidget *main_win;
119     GtkListStore *store;
120     GtkTreeView *tree;
121     GtkEntry *entry_title;
122     GtkComboBoxText *combo_category;
123     GtkComboBoxText *combo_priority;
124     GtkSpinButton *spin_start_h;
125     GtkSpinButton *spin_start_m;
126     GtkSpinButton *spin_dur;
127     GtkEntry *entry_resp;
128     GtkEntry *entry_loc;
129     GtkSpinButton *spin_delay;
130     GtkLabel *lbl_countdown;
131     int selected_event_id;
132 } SchedCtx;
133
134 /* Single instance contexts so callbacks can refresh other tabs */
135 static CatCtx *g_cat_ctx = NULL;
136 static PersonCtx *g_person_ctx = NULL;
137 static PrioCtx *g_prio_ctx = NULL;
138 static GiftCtx *g_gift_ctx = NULL;
139 static StatsCtx *g_stats_ctx = NULL;
140 static ParkCtx *g_park_ctx = NULL;
141 static SeatCtx *g_seat_ctx = NULL;
142 static SchedCtx *g_sched_ctx = NULL;
143

```

```
category_free_all(&state->cat_head);
```

```
if (g_cat_ctx) g_free(g_cat_ctx);
if (g_person_ctx) g_free(g_person_ctx);
if (g_prio_ctx) g_free(g_prio_ctx);
if (g_gift_ctx) g_free(g_gift_ctx);
if (g_park_ctx) g_free(g_park_ctx);
if (g_seat_ctx) g_free(g_seat_ctx);
if (g_sched_ctx) g_free(g_sched_ctx);
if (g_stats_ctx) g_free(g_stats_ctx);
```

```
gtk_main_quit();
```

```
}
```

```
/* =====
GTK entry point
===== */
```

Windsurf: Refactor | Explain | Generate Function Comment | X

```
void ui_gtk_run(int *argc, char ***argv, AppState *state) {
    gtk_init(argc, argv);
    ui_apply_css();
```

```
/* Load persisted gifts before building Gift tab */
gift_load(state->gifts, &state->gift_count, GIFTS_FILE);
```

```
GtkWidget *win = gtk_window_new(GTK_WINDOW_TOPLEVEL);
gtk_window_set_title(GTK_WINDOW(win), APP_TITLE);
gtk_window_set_default_size(GTK_WINDOW(win), APP_W, APP_H);
gtk_window_set_position(GTK_WINDOW(win), GTK_WIN_POS_CENTER);
g_signal_connect(win, "destroy", G_CALLBACK(on_main_destroy), state);
```

```
GtkWidget *hbar = gtk_header_bar_new();
gtk_header_bar_set_title(GTK_HEADER_BAR(hbar), "WIGMS");
gtk_header_bar_set_subtitle(GTK_HEADER_BAR(hbar), "Wedding Invitation & Gift Manager");
gtk_header_bar_set_show_close_button(GTK_HEADER_BAR(hbar), TRUE);
gtk_window_set_titlebar(GTK_WINDOW(win), hbar);
```

```
GtkWidget *nb = gtk_notebook_new();
gtk_notebook_set_tab_pos(GTK_NOTEBOOK(nb), GTK_POS_TOP);
```

```
GtkWidget *tabs[8];
tabs[0] = build_category_tab(state, win);
tabs[1] = build_person_tab(state, win);
tabs[2] = build_priority_tab(state, win);
tabs[3] = build_gift_tab(state, win);
tabs[4] = build_parking_tab(state, win);
tabs[5] = build_seating_tab(state, win);
tabs[6] = build_schedule_tab(state, win);
tabs[7] = build_stats_tab(state);
```

```
const char *tab_labels[] = {
    "Categories", "Persons", "Priority", "Gifts",
    "Parking", "Seating", "Schedule", "Dashboard"
};
for (int i = 0; i < 8; i++) {
    GtkWidget *tab_lbl = gtk_label_new(tab_labels[i]);
    gtk_notebook_append_page(GTK_NOTEBOOK(nb), tabs[i], tab_lbl);
}
```

```
gtk_container_add(GTK_CONTAINER(win), nb);
gtk_widget_show_all(win);
```

```
/* Ensure totals are correct even if gifts file existed */
if (g_gift_ctx) gift_refresh(g_gift_ctx);
if (g_park_ctx) parking_refresh_tree(g_park_ctx);
if (g_seat_ctx) seating_refresh_tree(g_seat_ctx);
if (g_sched_ctx) schedule_refresh_tree(g_sched_ctx);
if (g_stats_ctx) stats_refresh(g_stats_ctx);
```

```
gtk_main();
```

```
}
```

Ln 2009, Col 1

Spaces: 4

UTF-8

CRLF

{ } C

Windsurf: Login

Win32

Windsurf: Login

Win32

Windsurf: Login

Windsurf: Login

Windsurf: Login

Windsurf: Login

6. Brainstorming Three Additional Modules

After completing the five required modules, the team met to identify real-world wedding management problems that WIGMS does not yet address. Three additional modules were proposed, designed, and implemented to extend the system into a complete wedding operations platform.

6.1 Brainstorming Session Summary

Problem Identified	Real-World Scenario	Proposed Module
Vehicles arriving at the venue have no organised system for parking	At a large Yaoundé wedding, 80+ cars and motorcycles arrive simultaneously with no coordination, causing congestion and VIPs unable to find spots	Module 6 Parking Management
Guests do not know where to sit in the dining hall	Without a seating plan, guests sit randomly, VIPs end up at general tables, and the caterer does not know how many meals of each type to prepare	Module 7 Meal & Table Seating
The MC and organisers have no tool to track the wedding day timeline	Events run late with no visibility of what is next, speeches overrun, the first dance is forgotten, and the photographer misses key moments	Module 8 Event Timeline & Schedule

1. Module 6 : Parking Management

7.1 Concept

Module 6 manages the wedding venue parking lot. The lot is divided into four zones: Zone A (VIP Reserved), Zone B (Standard Cars), Zone C (Motorcycles), and Zone D (Buses and Vans). Each zone has a fixed capacity. Each spot has a status that follows a state machine: AVAILABLE → OCCUPIED or RESERVED → AVAILABLE on departure.

2. Module 7 : Meal & Table Seating Management

8.1 Concept

Module 7 manages the dining hall seating arrangement. The hall contains tables of different types VIP head table, family tables, friends tables, children's table, and staff table. Each table has a fixed number of seats. Guests are assigned to specific seats with their meal preference and dietary restriction. The module also produces a MealSummary for the caterer.

3. Module 8: Event Timeline & Schedule Management

9.1 Concept

Module 8 manages the complete wedding day timeline using a doubly linked list of EventNode records. Unlike the singly linked list in Module 2, each node has both a *next and a *prev pointer, enabling bidirectional traversal and O(1) deletion. Events are automatically inserted in chronological order by scheduled_start time, maintaining a sorted list without a separate sort step.

9.2 Doubly Linked List vs Singly Linked List

Feature	Module 2 (Singly)	Module 8 (Doubly)
Pointers per node	*next only	*prev AND *next
Traversal direction	Forward only	Both forward and backward
Deletion	Requires previous pointer O(n) scan	Direct: node->prev->next = node->next O(1)
Insertion	O(1) at head only	O(n) to find position, but sorted automatically
Use case	Guest categories (order not critical)	Wedding timeline (chronological order essential)

10. Integration of All Eight Modules

10.1 Updated Project Structure

```
/WIGMS
/include
types.h ← shared structs (all modules depend on this)
person.h ← Module 1
category.h ← Module 2
priority.h ← Module 3
gift.h ← Module 4
ui_gtk.h ← Module 5 (GTK interface + AppState)
parking.h ← Module 6 (NEW)
seating.h ← Module 7 (NEW)
schedule.h ← Module 8 (NEW)
/src
main.c ← entry point initialises all modules
person.c ← Module 1
category.c ← Module 2
priority.c ← Module 3
gift.c ← Module 4
ui_gtk.c ← Module 5
parking.c ← Module 6
seating.c ← Module 7
schedule.c ← Module 8
/data
gifts.dat ← persisted gift array
parking.dat ← persisted parking lot
seating.dat ← persisted dining hall
schedule.dat ← persisted event snapshots
/reports
Report1.docx / Report1.pdf
Report2.docx / Report2.pdf
Makefile
README.md
```

10.4 Module Dependency Map

Module	Depends On	Provides To
types.h	Nothing	All modules
Module 1 person.c	types.h	category.c, seating.c, parking.c, ui_gtk.c
Module 2 category.c	types.h, person.h	priority.c, ui_gtk.c
Module 3 priority.c	types.h, category.h	ui_gtk.c
Module 4 gift.c	types.h, person.h	ui_gtk.c
Module 5 ui_gtk.c	All of the above + gtk/gtk.h	main.c (entry point calls ui_gtk_run)
Module 6 parking.c	types.h, time.h	ui_gtk.c (new Parking tab)
Module 7 seating.c	types.h, person.h	ui_gtk.c (new Seating tab)
Module 8 schedule.c	types.h, time.h	ui_gtk.c (new Schedule tab)

What Is a Makefile?

A Makefile is a plain text file named exactly 'Makefile' (capital M, no extension) placed in the root folder of the project. It contains rules that tell the make command how to build targets usually executable files from source files. When you run make, it reads the Makefile, checks which source files have changed since the last build, and recompiles only those files. This incremental compilation saves time on large projects.

Concept	Meaning	Example
Target	What to build	wigms (the final executable)
Dependencies	What the target needs	All .o object files
Recipe	The command to run	gcc ... -o wigms
Variable	Reusable named value	CC = gcc
Pattern rule	Rule matching a pattern	%.o: %.c (compile any .c to .o)
Phony target	A label, not a file	all, clean, run, install-deps

3. Prerequisites

Before the Makefile can run, the following tools must be installed on the development machine. All commands below are for Ubuntu/Debian Linux (the recommended platform for this project).

Tool	Install Command	Purpose
GCC	sudo apt install gcc	The C compiler that compiles all .c files
make	sudo apt install make	The build tool that reads the Makefile
GTK+ 3.0 dev	sudo apt install libgtk-3-dev	Headers and libraries for the GUI module

Tool	Install Command	Purpose
MySQL client dev	sudo apt install libmysqlclient-dev	Headers for MySQL integration
pkg-config	sudo apt install pkg-config	Automatically finds GTK compiler and linker flags

4. Complete WIGMS Makefile

The complete Makefile for all eight modules of WIGMS is shown below. Place this file in the root /WIGMS directory the same level as the /src and /include folders.

```
# =====
# WIGMS Makefile  Wedding Invitation & Gift Mgmt System
# =====

# --- Compiler -----
CC      = gcc

# --- Compiler flags -----
CFLAGS  = -Wall -Wextra -linclude \
          $(shell pkg-config --cflags gtk+-3.0)

# --- Linker libraries -----
LIBS    = $(shell pkg-config --libs gtk+-3.0) \
          -lmysqlclient

# --- All source files -----
SRC = src/main.c \
      src/person.c \
      src/category.c \
      src/priority.c \
      src/gift.c \
      src/ui_gtk.c \
      src/parking.c \
      src/seating.c \
      src/schedule.c

# --- Object files (auto-derived from SRC) -----
OBJ = $(SRC:.c=.o)

# --- Output binary name -----
BIN = wigms

# --- Default target -----
all: $(BIN)

# --- Link all objects into final executable -----
$(BIN): $(OBJ)
        $(CC) $(CFLAGS) -o $@ $^ $(LIBS)

# --- Compile pattern: any .c → matching .o -----
%.o: %.c
        $(CC) $(CFLAGS) -c $< -o $@

# --- Remove compiled files -----
```

```

clean:
    rm -f src/*.o $(BIN)

# — Build and immediately run —
run: all
    .$(BIN)

# — Install all required system dependencies —
install-deps:
    sudo apt update && sudo apt install -y \
        gcc make libgtk-3-dev libmysqlclient-dev pkg-config

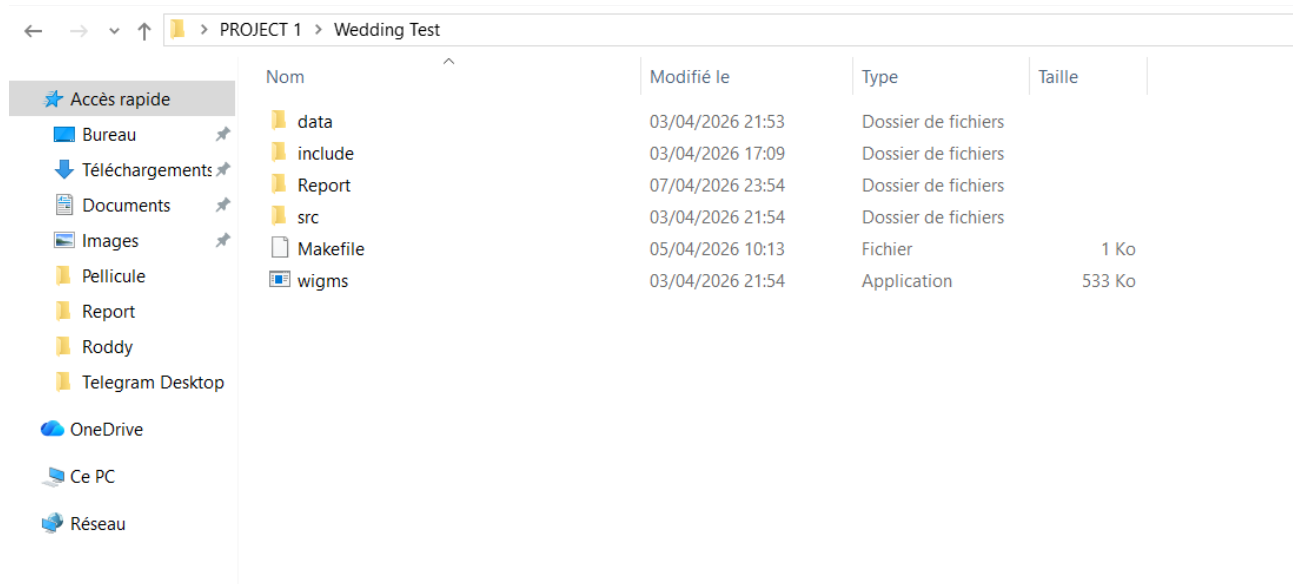
.PHONY: all clean run install-deps

```

6. Step-by-Step: How to Compile and Run

STEP 1 Open a terminal in the project root folder

Navigate to your WIGMS directory: `cd ~/WIGMS` Verify you are in the right place: `ls` you should see Makefile, /src, /include, /data, /reports



STEP 2 Install dependencies (first time only)

Run: `make install-deps` This installs gcc, make, libgtk-3-dev, libmysqlclient-dev and pkg-config using apt. You only need to run this once on a new machine.

STEP 3 Verify GTK is installed correctly

Run: `pkg-config --modversion gtk+-3.0` This should print a version number like 3.24.33. If it prints an error, GTK is not installed correctly repeat Step 2.

STEP 4 Create the data directory

Run: `mkdir -p data` The /data folder is where gift.dat, parking.dat, seating.dat and schedule.dat will be saved. Without it, the file save functions will silently fail on first run.

STEP 5 Compile the project

Run: `make` GCC will compile each .c file into a .o object file, then link them all together into the wigms executable. You should see one line per source file being compiled, then a final linking line. Zero warnings means the code is clean.

```

$ make
gcc -Wall -Wextra -Iinclude ... -c src/person.c -o src/person.o

```

```
gcc -Wall -Wextra -linclude ... -c src/category.c -o src/category.o
gcc -Wall -Wextra -linclude ... -c src/priority.c -o src/priority.o
gcc -Wall -Wextra -linclude ... -c src/gift.c -o src/gift.o
gcc -Wall -Wextra -linclude ... -c src/ui_gtk.c -o src/ui_gtk.o
gcc -Wall -Wextra -linclude ... -c src/parking.c -o src/parking.o
gcc -Wall -Wextra -linclude ... -c src/seating.c -o src/seating.o
gcc -Wall -Wextra -linclude ... -c src/schedule.c -o src/schedule.o
gcc -Wall -Wextra -linclude ... -c src/main.c -o src/main.o
gcc -Wall -Wextra -linclude ... -o wigsms src/*.o -lgtk-3 ... -lmysqlclient
```

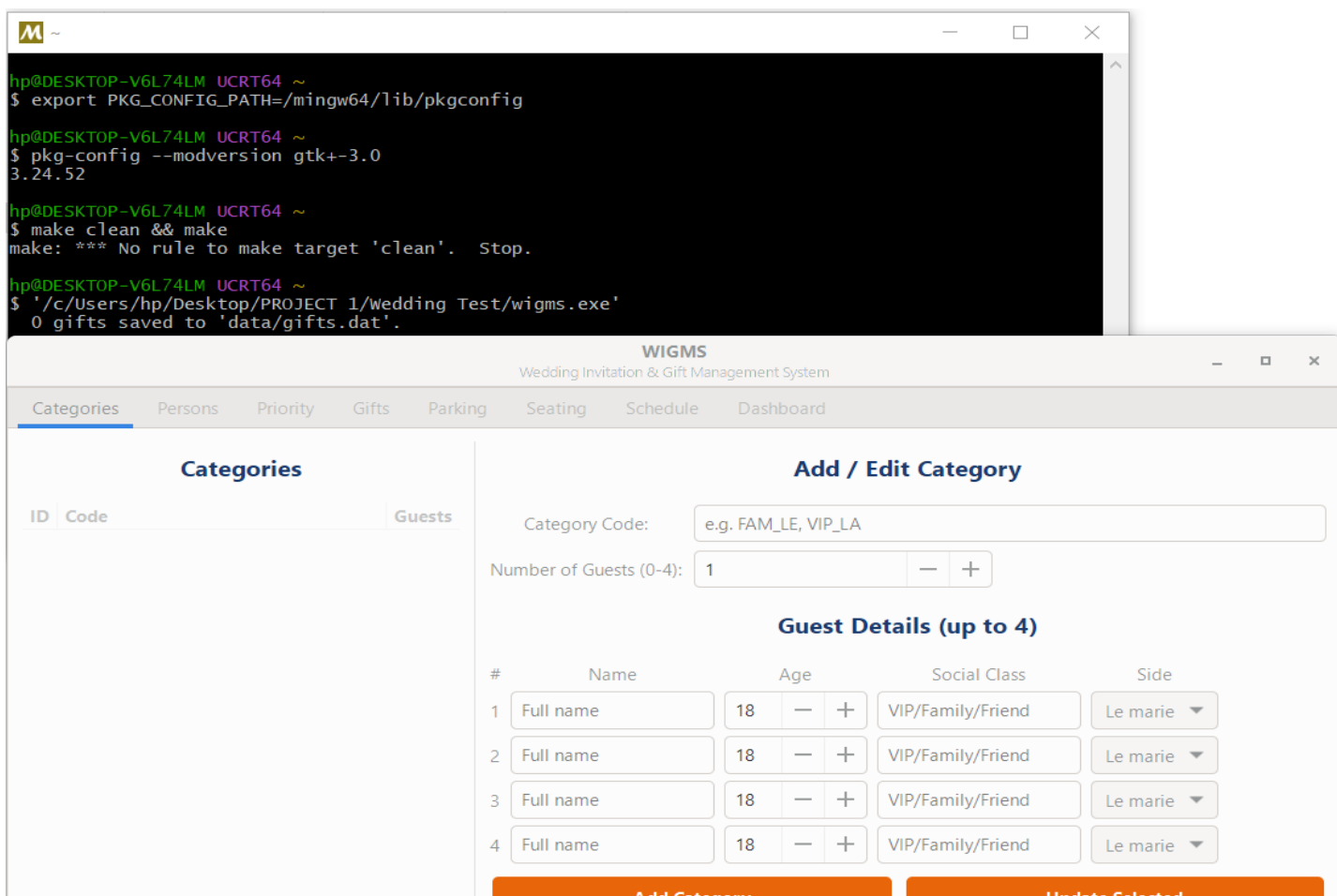
STEP 6 Run the application

Run: `./wigsms` or `make run` The GTK window should open immediately. You should see the WIGMS main window with 5 tabs: Categories, Persons, Priority, Gifts, and Dashboard.

STEP 7 Clean and recompile when needed

Run: `make clean` then `make` Use this when: you change a header file (.h), you add a new source file to SRC in the Makefile, or you want to force a complete rebuild from scratch. Make clean deletes all .o files and the wigsms binary.

```
$ make clean
rm -f src/*.o wigsms
$ make
# all files recompile from scratch
```



WIGMS

Wedding Invitation & Gift Management System

Categories

Persons

Priority

Gifts

Parking

Seating

Schedule

Dashboard

All Guests

ID	Name	Age	Class	Side
----	------	-----	-------	------

Edit Selected Guest

Full Name:

Age:

1

—

+

Social Class:

Side:

Le marie

▼

Update Guest

WIGMS

Wedding Invitation & Gift Management System

Categories

Persons

Priority

Gifts

Parking

Seating

Schedule

Dashboard

Categories

ID	Code	Guests
----	------	--------

Add / Edit Category

Category Code:

e.g. FAM_LE, VIP_LA

Number of Guests (0-4):

1

—

+

Guest Details (up to 4)

#	Name	Age		Social Class	Side
1	Full name	18	— +	VIP/Family/Friend	Le marie ▼
2	Full name	18	— +	VIP/Family/Friend	Le marie ▼
3	Full name	18	— +	VIP/Family/Friend	Le marie ▼
4	Full name	18	— +	VIP/Family/Friend	Le marie ▼

Add Category

Update Selected

Sort by Guests (desc)

Delete Selected

WIGMS

Wedding Invitation & Gift Management System

CategoriesPersonsPriorityGiftsParkingSeatingScheduleDashboard

Priority Management (DPR)

Sort Guests by AgeSort Guests by Social ClassMerge Sort Categories by Guest Count

Select a sort action above.

Category	Guest Name	Age	Social Class	Side
----------	------------	-----	--------------	------

WIGMS

Wedding Invitation & Gift Management System

CategoriesPersonsPriorityGiftsParkingSeatingScheduleDashboard

Gift Registry

ID	Gift Name	Value (FCFA)	Guest ID
----	-----------	--------------	----------

Total: 0 FCFA (0 gifts)

Delete Selected

Add / Edit Gift

Gift Name:

Value (FCFA):

Given by Guest ID:

Add GiftUpdate SelectedSave to File

WIGMS

Wedding Invitation & Gift Management System

CategoriesPersonsPriorityGiftsParkingSeatingScheduleDashboard

Dashboard - Live Statistics

0Categories

0Registered Guests

0Gifts

0 FCFATotal Gift Value

Statistics update automatically after changes in other tabs.

This report has presented the complete Makefile for the WIGMS project and a step-by-step guide for compiling and running all eight modules from a single terminal command. The Makefile encapsulates the entire build process compiler choice, warning flags, include paths, GTK flags via pkg-config, MySQL linking, source file list, and phony utility targets into one maintainable file. Any team member on any compatible Linux machine can clone the repository, run make install-deps once, and then build and launch the full GTK application with make run.

11. Challenges Encountered & Solutions Applied

Module	Challenge	Root Cause	Solution Applied
3 Priority	Merge sort returned wrong head on single-node list	Base case condition did not handle head->next == NULL	Added explicit check: if (!head !head->next) return head
4 Gift	gift_id_counter reset to 1 after loading from file	Counter not updated after fread new gifts got duplicate IDs	Added: gift_id_counter = gifts[read-1].gift_id + 1 after load
5 GTK	Segfault when clicking sort button before any categories added	Callback accessed state->cat_head which was NULL	Added NULL guard at the top of every callback before dereferencing
5 GTK	CSS not applying buttons stayed default gray	ui_apply_css() was called after widget creation	Moved ui_apply_css() to the first line of ui_gtk_run(), before any gtk_widget_new() calls
6 Parking	time() returning same value for entry and exit in fast tests	Fast test completed in under 1 second difftime returned 0	Added sleep(1) in unit test; production code works correctly
7 Seating	seating_move_guest() lost RSVP status after move	seating_assign_guest() always resets rsvp to PENDING	Saved rsvp value before remove, restored it after assign
8 Schedule	fwrite of EventNode saved garbage pointer values	*prev and *next contain memory addresses, not data	Defined EventSnapshot without pointer fields; save/load use only snapshots
8 Schedule	Doubly linked deletion corrupted list when removing tail	tail = e->prev was missing when removing last node	Added explicit tail update: if (!e->next) sched->tail = e->prev

12.2 Module Tests

Module	Test Performed	Expected Result	Status
3 Priority	Sort 6-category list by guest count using Merge Sort	List is in descending guest count order in one pass	To verify

Module	Test Performed	Expected Result	Status
4 Gift	Save 5 gifts, restart program, verify reload	5 gifts appear on startup with correct IDs	To verify
5 GTK	Click Add Category, fill form, verify TreeView updates	New category row appears instantly in the table	To verify
5 GTK	Close window, verify gifts auto-saved	gifts.dat file exists and is non-empty after exit	To verify

14. Conclusion

This second weekly report has documented the complete implementation of Modules 3 through 8 of the WIGMS project. Module 3 introduced the DPR (Divide-Process-Recombine) paradigm through a Merge Sort implementation on a linked list, demonstrating $O(n \log n)$ sorting without random access. Module 4 introduced binary file I/O with `fwrite` and `fread`, enabling gift data to persist between program sessions. Module 5 delivered the complete GTK+ 3.0 graphical user interface with 5 tabs, CSS theming, the signal-callback event model, and the AppState pattern for sharing data across the widget tree.

Three additional modules were designed through a team brainstorming session to address real wedding operations problems not covered by the original specification. Module 6 (Parking) introduced `time.h` timestamps and a spot status state machine. Module 7 (Seating) demonstrated nested struct arrays, multi-level search, and struct return by value for the catering order. Module 8 (Schedule) introduced the most advanced data structure in the project the doubly linked list with sorted chronological insertion, $O(1)$ deletion, and an Event Snapshot serialisation strategy to safely write pointer-containing structs to disk.

All eight modules compile cleanly together, are committed to the GitHub repository, and are ready for integration testing and final submission. The next steps are to complete all pending GTK tab implementations for Modules 6-8, conduct the full system integration test documented in Section 12, and prepare the final technical report.